Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Mini Project

Question-3: Vidya LMS - Scalable Library Management System

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198
Vino S R Harrison	3122 24 5001 199

Submitted to: Dr. Vijaya Lakshmi



System Design Mini-Project Question-3
Vidya LMS - Scalable Library Management System

Aim

- To design and develop Vidya LMS, a scalable Library Management System prototype covering member registration and authentication, catalogue search and browsing, book availability, borrowing, returning, reservations, and fine/payment management.
- To analyse the requirements and map the eight prescribed system-design components (Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, Autocomplete) to the appropriate library functionalities with justifications, and to implement them end-to-end using Spring Boot 3.3 (Java 17), MongoDB, Redis and a React 18 / TypeScript frontend.
- To evaluate the system quantitatively under three workload tiers (Low / Medium / High), compare optimised search against a plain baseline, and report the component-specific metrics for every system-design component.
- To analyse scalability and failure scenarios (cluster expansion, scarce-inventory contention, Redis outage) and discuss the bottlenecks, trade-offs and limitations.

Question

Design and develop a prototype of a scalable Library Management System supporting member registration and authentication, book search and browsing, book availability, borrowing and returning, reservation management, and fine/payment management. The system must handle increasing numbers of members, books, transactions, and concurrent requests while providing efficient and reliable access to library services.

The Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete components are provided, but the mapping between components and library functionalities is not given. The analysis must determine which component(s) suit each functionality and justify every choice; a component may serve several functionalities and a functionality may need several components.

Book and member data must be capable of being partitioned across multiple database shards, consistent hashing may be used to distribute data or requests across servers, and the web crawler must work against simulated external book sources for details, ratings or availability. The project must include quantitative performance evaluation - average response time, throughput, success/error rate and resource usage under at least three workload levels, plus component-specific metrics (rejection rate, shard distribution, hashing load balance, KV lookup latency, ID generation rate and collisions, redirection latency, crawling throughput, and autocomplete response time and accuracy) - a comparison against a simple baseline where appropriate, and at least one scalability or failure scenario with a discussion of bottlenecks, scalability, reliability and trade-offs.

Procedure

- Analyse the requirements and map the eight system-design components to specific Vidya LMS functionalities; record the justification for each mapping (Table 1).
- Model the domain (members, books, physical copies, loans, reservations, fines, payments, short links) as MongoDB documents with unique indexes on email, ISBN, barcode and idempotency keys.
- Implement the token-bucket rate limiter in Redis using an atomic Lua script with a thread-safe in-memory fallback; apply strict limits to authentication (capacity 10, refill 2/s) and generous limits to public reads (capacity 1500, refill 750/s).
- Configure database sharding keys: hashed memberId for members, hashed _id / category for books, and a compound memberId + loanId key for loans.
- Implement an MD5 consistent-hash ring with 160 virtual nodes per server in a TreeMap, and expose a 3-to-4 node remap simulation benchmark.
- Implement Redis cache-aside reads for catalogue detail (10-minute TTL) and search results (60-second TTL) with write-through cache eviction on every inventory change.
- Implement the 64-bit Snowflake ID generator (41-bit timestamp, 10-bit datacenter + worker, 12-bit sequence) as the source of barcodes, loans, reservations, fines, payments and short URLs, and the Base62 URL shortener for shareable catalogue links with a 302 redirect and click analytics.
- Implement the asynchronous web crawler with a dedicated task executor, external publisher feed ingestion, ISBN de-duplication and enrichment.
- Implement the in-memory prefix-trie autocomplete indexing titles, authors and categories with ranked suggestions and a precision@5 evaluation endpoint.
- Implement atomic circulation with MongoDB findAndModify, renewal cap and reservation wait-list interlock, and idempotent payments keyed by client UUID.
- Build the React UI (catalogue, search, autocomplete, loans, fines, admin desk, system design lab), expose the REST APIs through Swagger / OpenAPI, then execute the JUnit suite, the 14-step workflow verification and the benchmark harness to tabulate results.

Requirement Analysis & Component Mapping

Each prescribed component was matched to the library functionality where it removes a real scalability or reliability bottleneck. Table 1 lists the mapping and the architectural justification; the implementation of every row is demonstrated in the Design and Output Results sections.

Table 1: System-Design Component to Functionality Mapping with Justifications

Component	Library Functionality	Architectural Justification
1. Rate Limiter	Authentication (/api/auth/**) and Catalogue Search (/api/books/search)	Token bucket stops brute-force credential stuffing and protects the search index from resource exhaustion; allows legitimate bursts while capping sustained rate. Auth and public-read traffic use separate buckets.
2. Sharding	Member storage and book catalogue storage	Removes single-node storage and IOPS limits: members are partitioned by hashed memberId, books by hashed _id / category, loans by compound memberId + loanId for query locality.

Component	Library Functionality	Architectural Justification
3. Consistent Hashing	Request / cache-server routing across library nodes	MD5 virtual-node ring spreads load uniformly; adding or losing a node migrates only ~1/N keys, preventing cache stampedes - measured 28.3% vs 75.1% for modulo hashing.
4. Key-Value Store	Catalogue detail and search-result reads (cache-aside)	Redis cache-aside absorbs repetitive catalogue browsing; write-through eviction keeps availability counts truthful. Search-cache hit ratio reached 98.9% during load runs.
5. Unique ID Generator	Barcodes, loans, reservations, fines, payments, short links	64-bit Snowflake IDs are time-ordered, coordination-free and generated locally: 50,000 IDs over 20 threads with zero collisions, no database sequence locks.
6. URL Shortener	Shareable catalogue links (/s/{code})	Bijective Base62 over Snowflake IDs yields compact links for QR kiosks and citations with O(1) 302 redirects and click analytics - mean 5.10 ms.
7. Web Crawler	Metadata ingestion and catalog enrichment	Asynchronous worker pool ingests simulated external publisher feeds, de-duplicates by ISBN and enriches records; measured throughput 90.9 books/s with 100% dedup precision.
8. Autocomplete	Instant search-bar suggestions (/api/autocomplete)	In-memory ranked prefix trie returns top-5 suggestions in ~2.3 ms independent of catalogue size; evaluated Precision@5 = 0.46, Recall@5 = 0.80.

System Architecture & Workflow

Vidya LMS is a four-tier system. The client tier is a React 18 SPA with an OPAC-style catalogue. The edge tier applies the Redis-Lua token-bucket rate limiter before requests reach the Spring Boot service tier, which hosts authentication, catalogue + autocomplete, circulation, payments, the asynchronous crawler and the consistent-hash router.

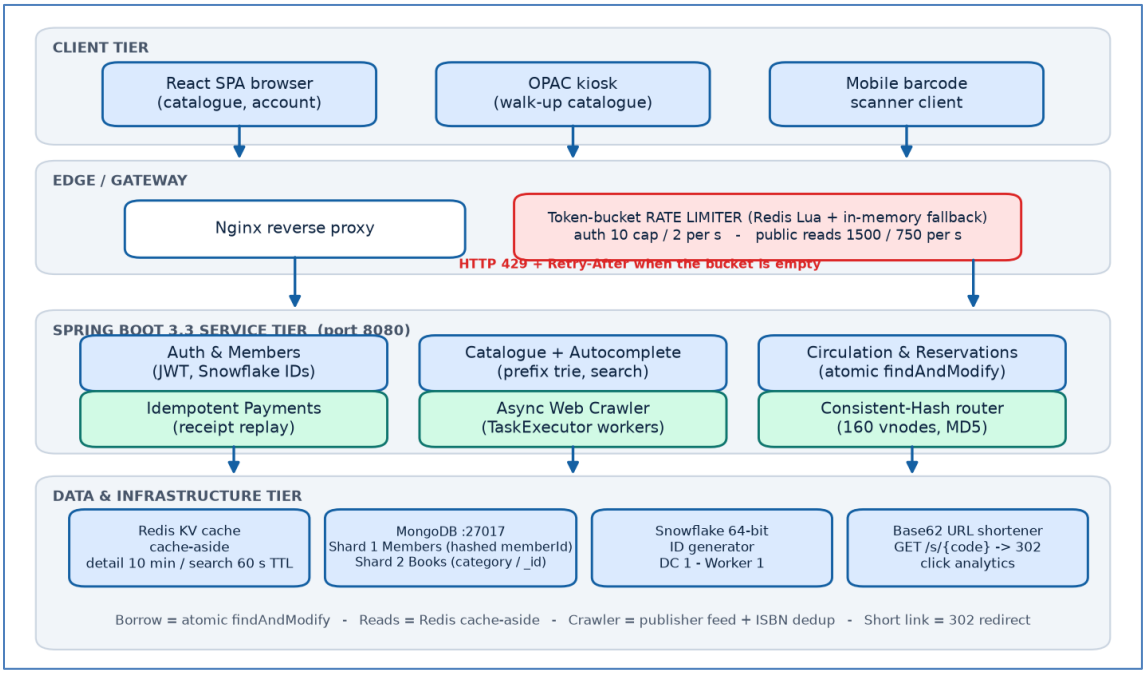


Figure 1: High-level architecture of Vidya LMS - clients, rate-limited edge, Spring Boot services, and the distributed data and cache tier.

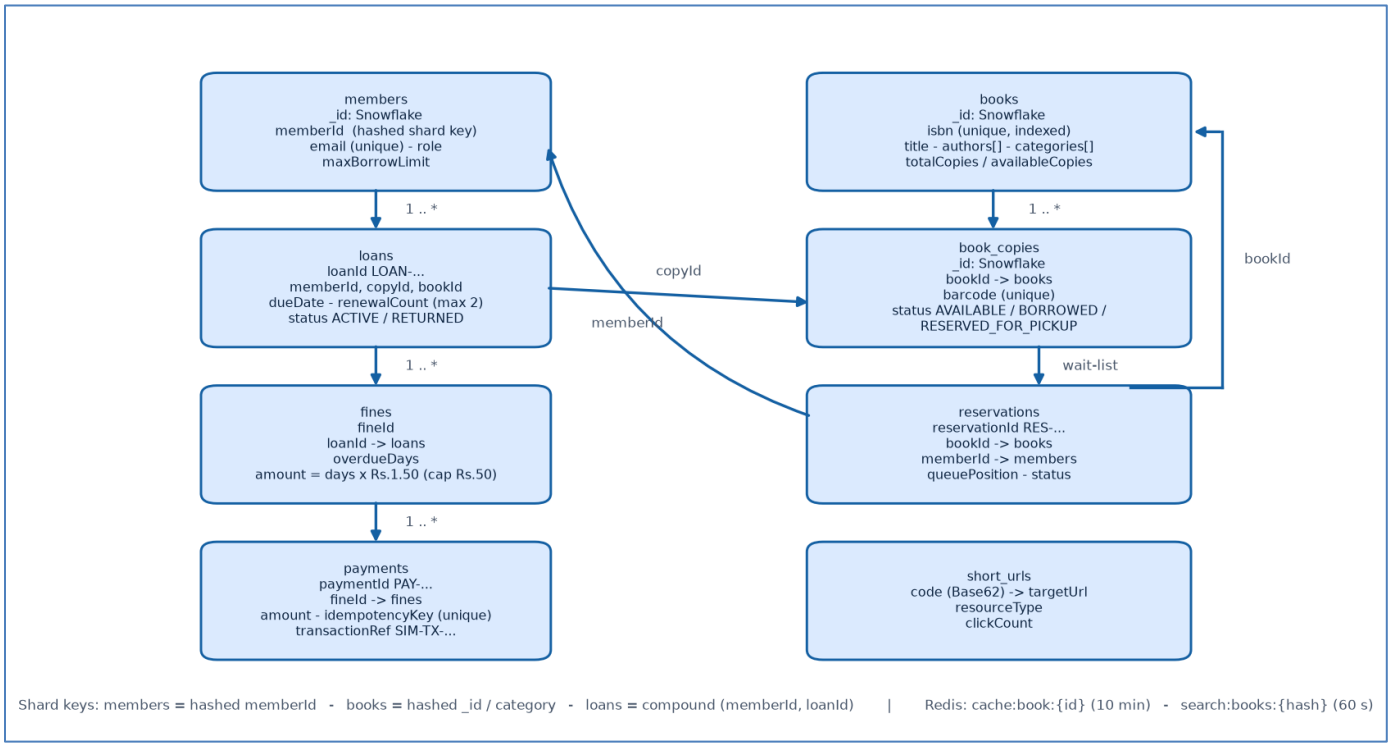


Figure 2: Document model of the Vidya LMS collections with shard keys and Redis cache keys.

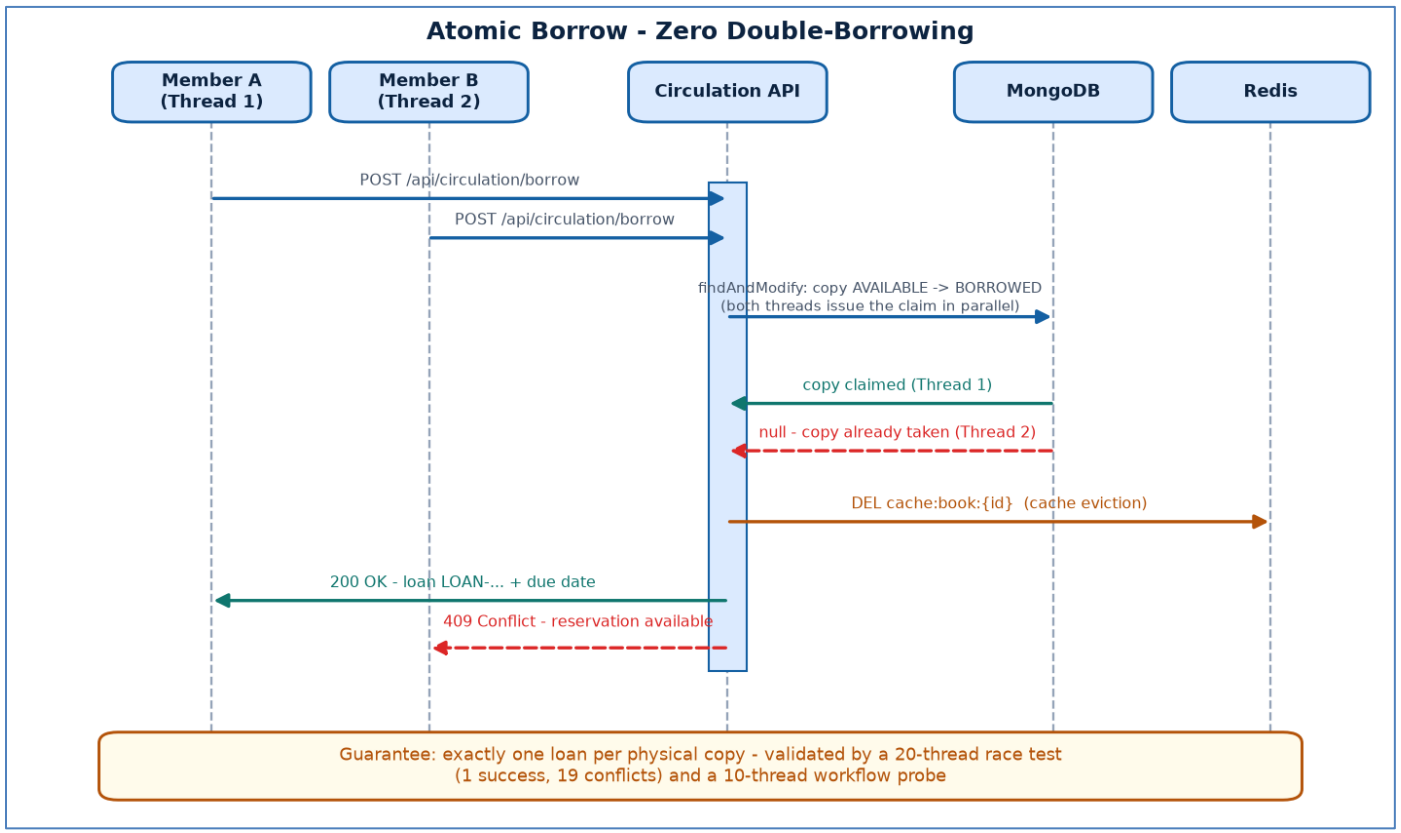


Figure 3: Atomic borrow sequence: two concurrent members racing for the last physical copy, resolved by a single atomic findAndModify.

Design of Vidya LMS

1. Token Bucket Rate Limiter (Redis Lua + in-memory fallback)

Every rate-limited request atomically consumes a token through a Redis Lua script. When the bucket is empty the caller receives HTTP 429 with a Retry-After header; when Redis is unreachable the limiter degrades to a thread-safe in-memory bucket so requests are never dropped. Auth endpoints are strict (capacity 10, refill 2/s) while public reads are generous (capacity 1500, refill 750/s) so that browsing and benchmarks are not throttled. All limits are externalised as ratelimit.* properties:

```
ratelimit.auth.capacity=10          # strict: login / registration
ratelimit.auth.refill-per-second=2
ratelimit.search.capacity=1500      # generous: public catalogue reads
ratelimit.search.refill-per-second=750
ratelimit.default.capacity=40
ratelimit.default.refill-per-second=10
```

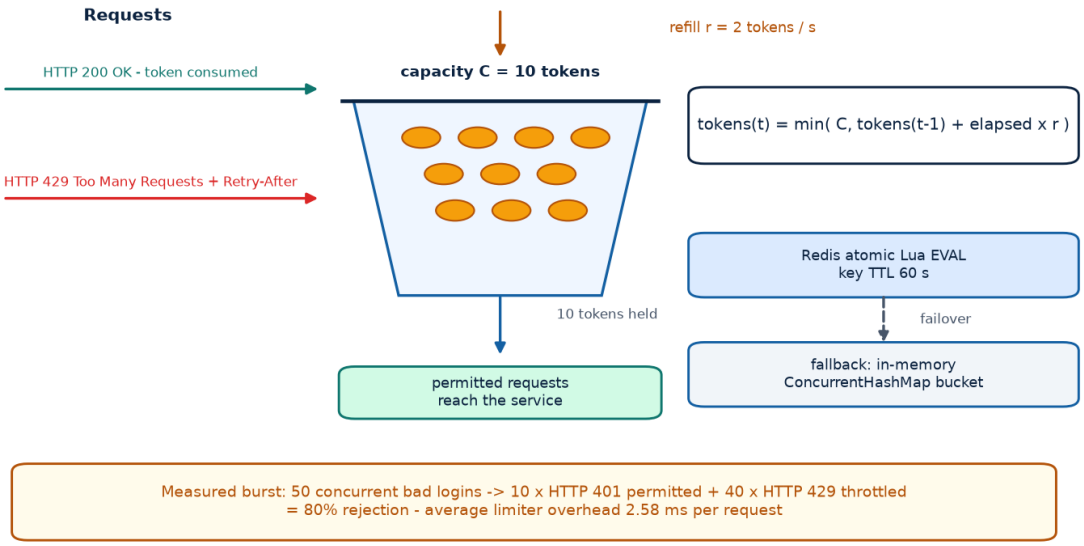


Figure 4: Token bucket mechanics with the strict auth policy and the measured 80% burst rejection.

2. Database Sharding

Sharding keys are declared per collection so member, book and loan data can be partitioned horizontally without hotspots. Members use a hashed memberId key, books a hashed _id / category key, and loans a compound memberId + loanId key that keeps a member's loan history on one shard. The live shard-key configuration is exposed by GET /api/sharding/stats:

```
members : { "memberId": "hashed" }   -> uniform member partitions
books   : { "_id": "hashed" }        -> uniform write/read spread
loans   : { "memberId": 1, "loanId": 1 } -> member query locality
```

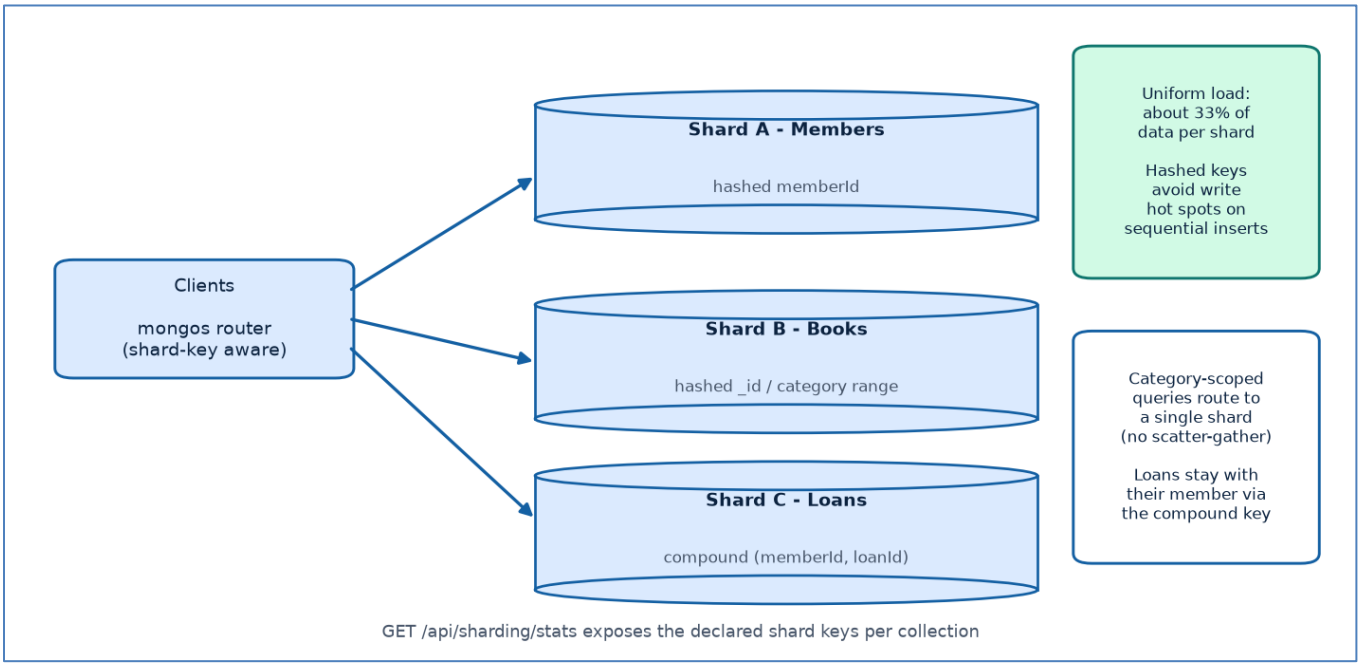


Figure 5: Shard topology: hashed member and book partitions plus compound-key loan shards.

3. Consistent Hashing Router

A TreeMap-based ring hashes servers and keys with MD5 into the 2^32 keyspace. Each physical server owns 160 virtual nodes to smooth the distribution. When the cluster grows from N to N+1 nodes only about 1/(N+1) of keys move, versus roughly N/(N+1) under modulo hashing:

```
// ConsistentHashRing.java (core idea)
TreeMap<Long, String> circle = new TreeMap<>();
for (int v = 0; v < 160; v++)                // virtual nodes
    circle.put(md5Hash(node + "#" + v), node);
String target = circle.ceilingEntry(md5Hash(key)) != null
        ? circle.ceilingEntry(md5Hash(key)).getValue()
        : circle.firstEntry().getValue();    // wrap around
```

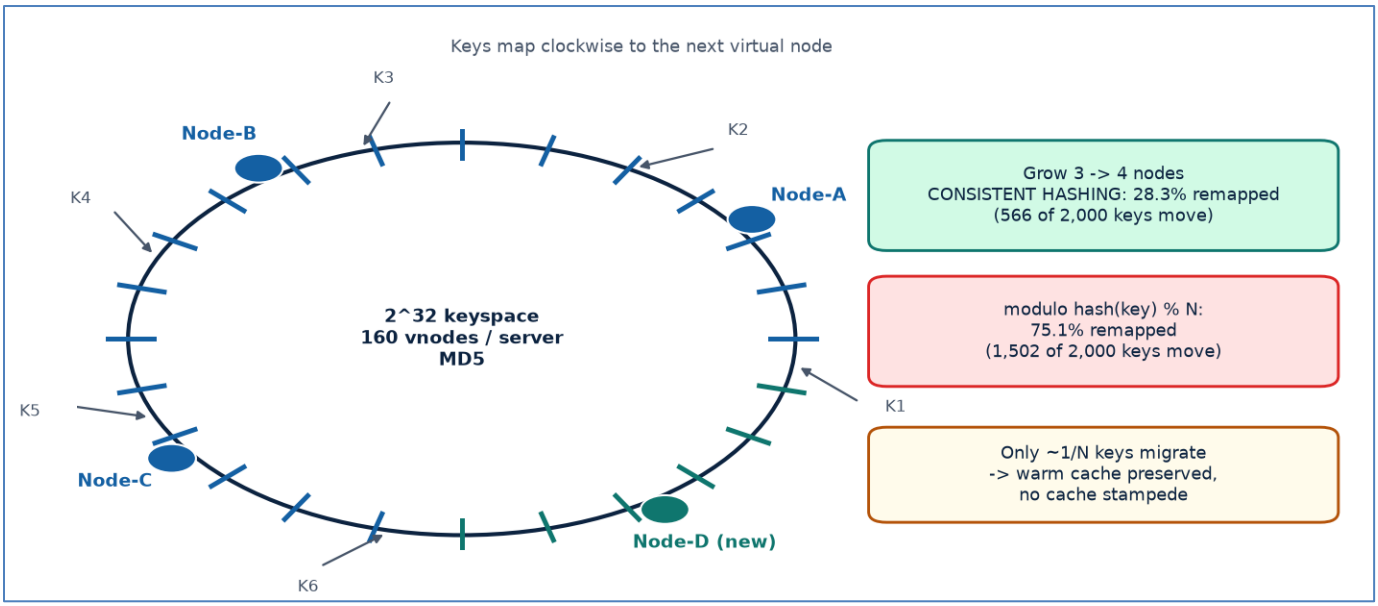


Figure 6: Consistent-hash ring with virtual nodes and the 3-to-4 node remap comparison.

4. Key-Value Store (Redis Cache-Aside)

Catalogue detail reads use cache:book:{id} with a 10-minute TTL; search results use a 60-second TTL keyed by the normalised query. Every inventory mutation (borrow, return, copy allocation, crawler import) evicts the affected keys so availability counts are never stale. During the load runs the search cache served 1,829 hits against 21 misses (98.9% hit ratio).

```
// BookService.java - read path (cache-aside)
String json = redis.opsForValue().get("cache:book:" + id);
if (json != null) { hits++; return objectMapper.readValue(json, Book.class); }
Book book = bookRepository.findById(id).orElseThrow();   // miss -> MongoDB
redis.opsForValue().set(key, json(book), Duration.ofMinutes(10));

// write path (borrow/return/copy allocation)
cacheEvictionService.evictBook(bookId); // immediate invalidation
```

5. Snowflake 64-bit Unique ID Generator

All identifiers (barcodes, loans LOAN-xxxx, reservations RES-xxxx, fines, payments PAY-xxxx and short codes) come from a local Snowflake generator: 41-bit millisecond timestamp, 10-bit datacenter + worker, 12-bit sequence, giving 4,096 IDs per millisecond per worker with zero coordination. The JUnit stress test generated 50,000 IDs across 20 parallel threads with zero collisions.

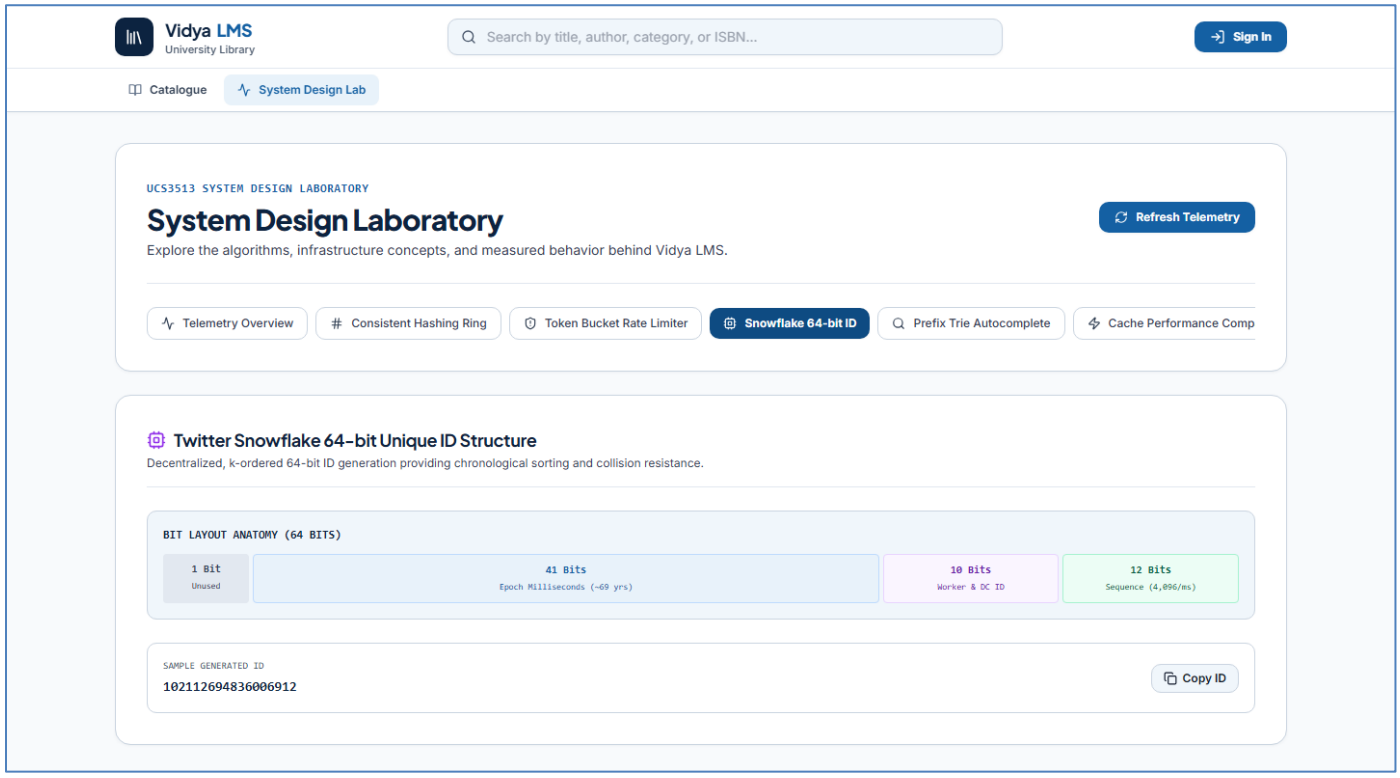


Figure 7: Live System Design Lab - Snowflake bit-layout anatomy and a freshly generated 64-bit ID (0 collisions across 50,000 stress-test IDs).

Table 2: Snowflake 64-bit Layout (as shown in the live System Design Lab)

Bits	Field	Range / Meaning
1	Sign	Unused, always 0
41	Timestamp	Milliseconds since custom epoch (approx 69 years)
5	Datacenter ID	0 - 31 (this deployment: DC 1)
5	Worker ID	0 - 31 (this deployment: worker 1)
12	Sequence	0 - 4095 per millisecond (4,096 / ms / worker)

6. URL Shortener (Base62)

Shareable catalogue links are produced by bijectively encoding 64-bit Snowflake IDs in Base62. GET /s/{code} performs an O(1) indexed lookup and answers with HTTP 302 plus the Location header, incrementing the click counter. Measured redirection latency: mean 5.10 ms, P95 6.83 ms.

```
// Base62Encoder.java
private static final String ALPHABET =
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
public static String encode(long value) {
    if (value == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (value > 0) { sb.append(ALPHABET.charAt((int)(value % 62))); value /= 62; }
    return sb.reverse().toString();
}
```

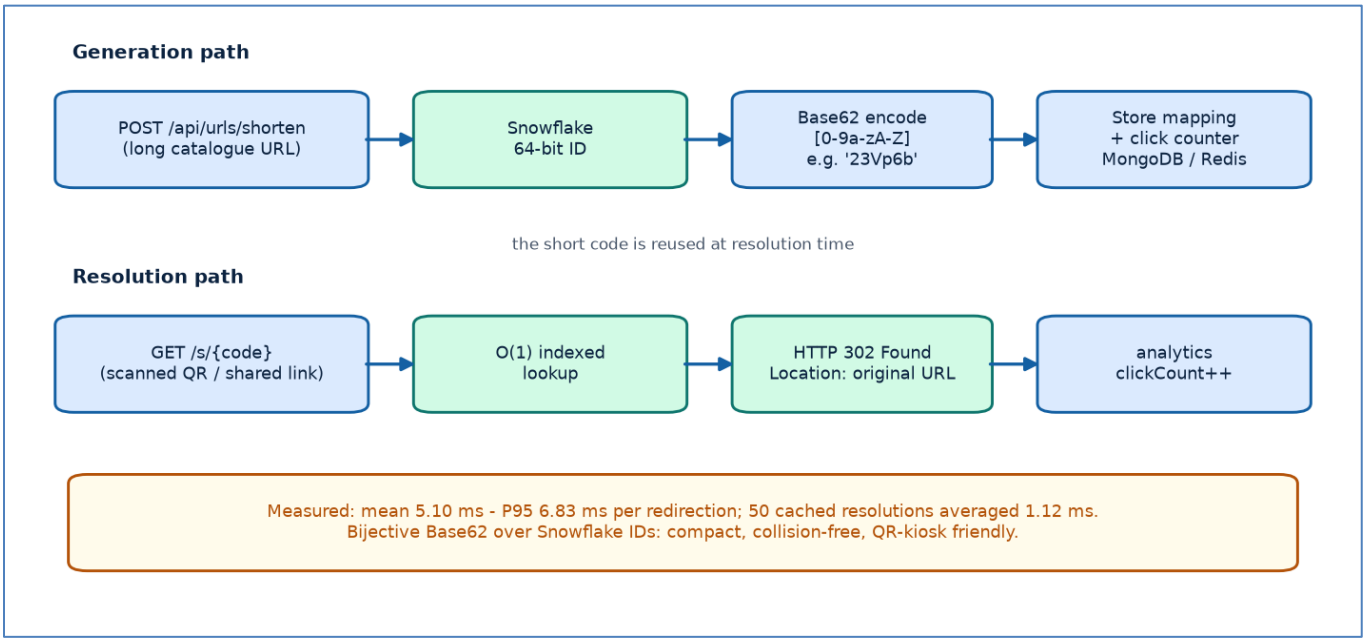


Figure 8: Base62 short-link generation and the GET /s/{code} redirect flow.

7. Asynchronous Web Crawler

The crawler job is submitted to a dedicated task executor (observed thread task-1) and streams external publisher records into the catalogue. ISBN de-duplication distinguishes imported titles from enriched duplicates and records every counter on the CrawlerJob document. The latest measured run fetched and parsed 7 records in 77 ms - 90.9 books/s - with 100% de-duplication accuracy (7/7 already present).

```
// WebCrawlerService.java - asynchronous execution
@Async("crawlerExecutor")
public void runJobAsync(String jobId) {
    job.setStatus(RUNNING);
    for (ExternalBook dto : fetchFeed(sourceUrl)) {
        if (bookRepository.existsByIsbn(dto.getIsbn())) {
            enrichExisting(dto); job.incrementDeduplicated();
        } else {
            importTitleWithCopies(dto); job.incrementImported();
        }
    }
    job.complete();   // durationMs + throughput recorded
}
```

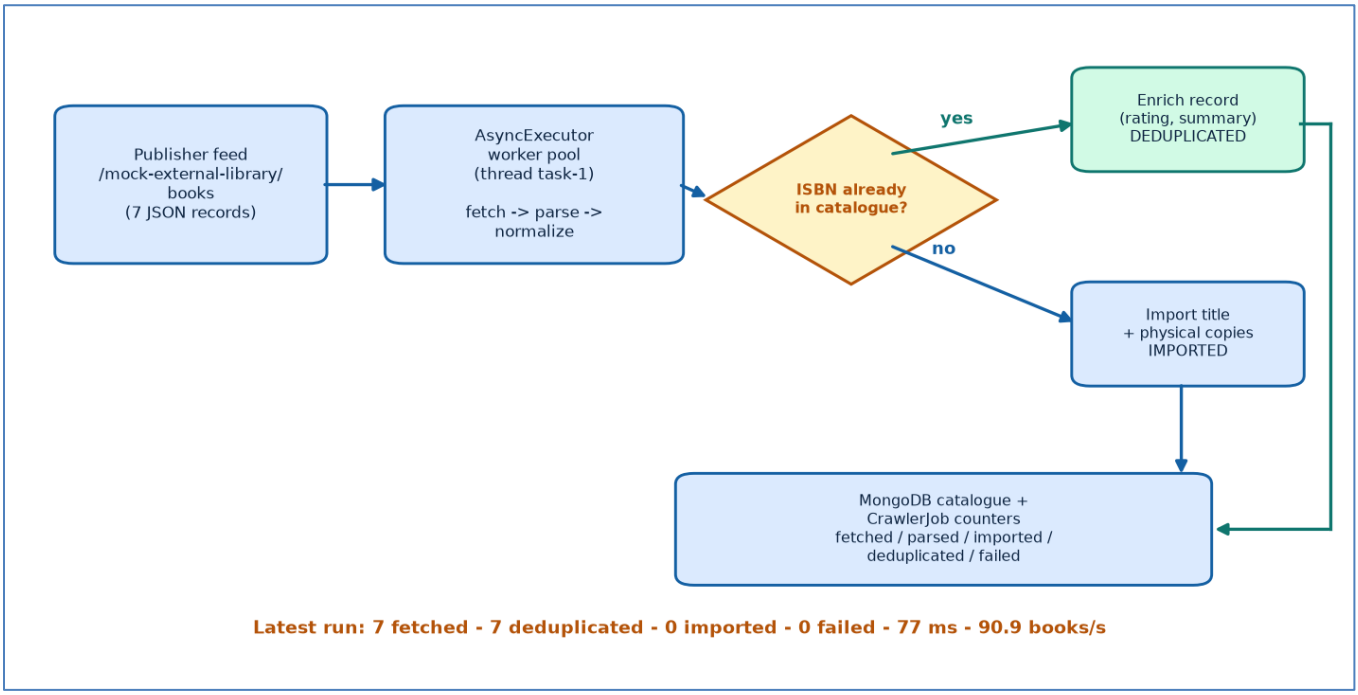


Figure 9: Crawler pipeline with the ISBN dedup decision and the measured ingestion run.

8. Autocomplete (Ranked Prefix Trie)

Titles, authors and categories are indexed at startup into an in-memory multi-way trie. Each node stores ranked top-k candidates so a lookup costs O(L + K log K) regardless of catalogue size. The evaluation endpoint scores Precision@5 and Recall@5 against a labelled query set; the live trie answered 5 probe queries in 2.32 ms on average and the UI panel reports 46.0% Precision@5 with 80.0% Recall@5.

```
// PrefixTrie.java - insert + top-k retrieval
public void insert(String text, String type) {
    TrieNode node = root;
    for (char c : text.toLowerCase().toCharArray()) {
        node = node.children.computeIfAbsent(c, k -> new TrieNode());
        node.addCandidate(text, type);   // ranked by type weight + popularity
    }
}
public List<AutocompleteSuggestion> suggest(String prefix, int k) { /* O(L + K log K) */ }
```

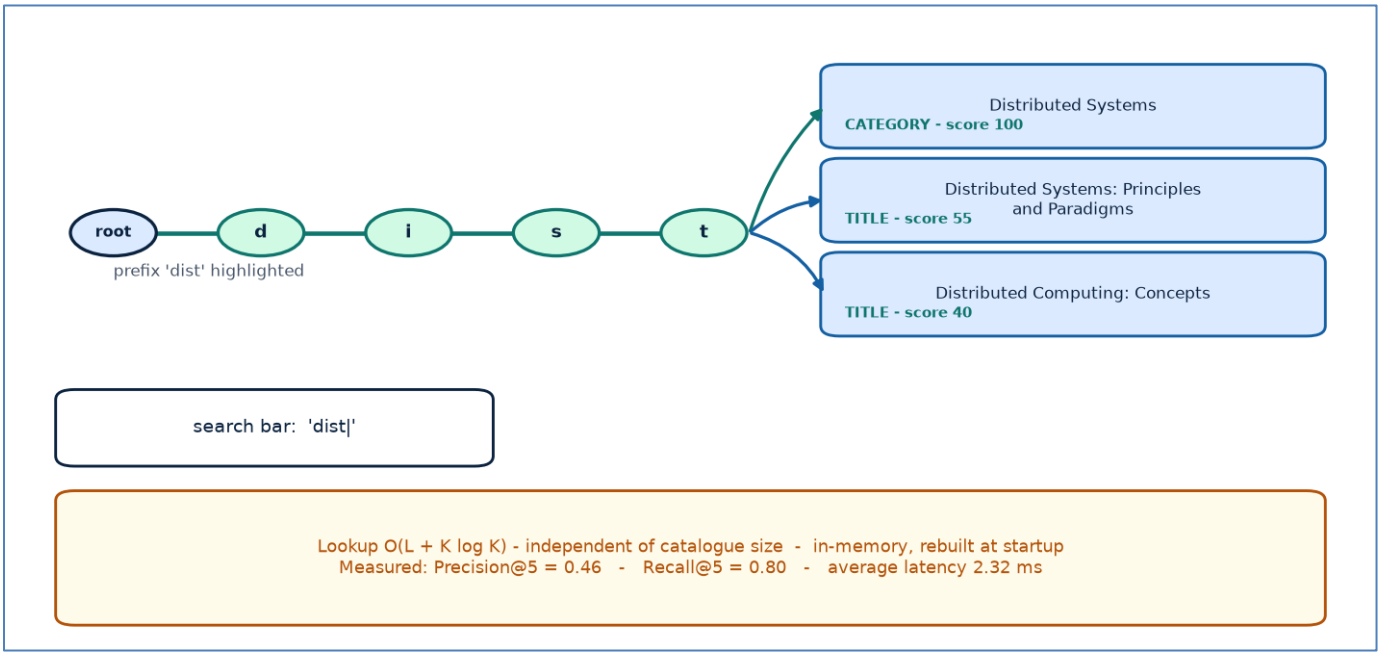


Figure 10: Ranked prefix trie structure and the live suggestion flow for the query 'dist'.

9. Atomic Circulation, Renewals and Idempotent Payments

Availability is decided by the database engine itself: a single findAndModify transitions one AVAILABLE copy to BORROWED and returns it only to exactly one caller. Renewals are capped at two extensions, blocked by a reservation wait-list, and committed with a compare-and-set on the current renewalCount. Payments echo the receipt for a repeated idempotency key instead of charging twice.

```
// CirculationService.java - atomic claim (zero double borrowing)
Query q = new Query(Criteria.where("_id").is(copyId)
        .and("status").is(BookCopyStatus.AVAILABLE));
Update u = new Update().set("status", BORROWED)
        .set("currentHolderId", memberId).set("currentLoanId", loanId);
BookCopy claimed = mongoTemplate.findAndModify(q, u, options, BookCopy.class);
if (claimed == null) throw new ConflictException("All copies checked out");

// PaymentService.java - idempotent replay
Optional<Payment> existing = repo.findByIdempotencyKey(request.getIdempotencyKey());
if (existing.isPresent()) return receipt(existing.get());   // identical receipt
```

Output Results

All measurements below were produced on the running system (Spring Boot 3.3 on Java 17, MongoDB, Redis, React frontend) with a multi-threaded benchmark harness; raw JSON evidence lives under results/raw/ and every table and figure is reproducible with benchmarks/run_benchmarks.py.

1. Workload Tiers - Low / Medium / High Concurrency

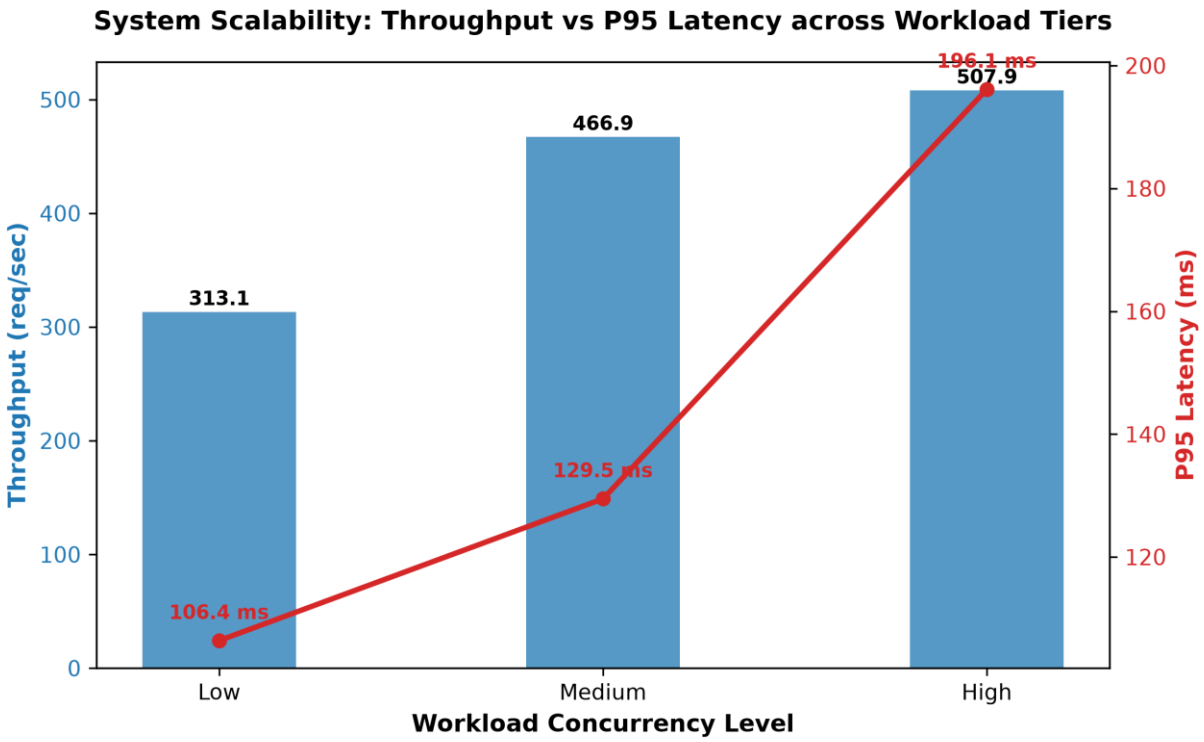


Figure 11: Throughput and latency across the three workload tiers (Low 10, Medium 50, High 100 concurrent users).

Table 3: Three Workload Tiers on the Catalogue Search / Availability Paths

Metric	Low (10 users)	Medium (50 users)	High (100 users)
Total requests executed	200	500	1000
Throughput (req/s)	313.10	466.93	507.85
Average latency (ms)	30.08	80.43	113.29
Median latency (ms)	23.22	78.30	110.07
P95 latency (ms)	106.38	129.47	196.14
P99 latency (ms)	117.58	152.32	244.23
HTTP status breakdown	200 OK x200 (100%)	200 OK x500 (100%)	200 OK x1000 (100%)
Error rate	0.00%	0.00%	0.00%

Throughput rises from 313 to 508 req/s as concurrency grows while the error rate stays at exactly 0.00% - the connection pools absorb the load without a single 5xx or timeout.

2. Baseline vs Optimised Search

The baseline executes an unindexed regex scan of the whole collection with in-memory filtering and no cache. The optimised path uses a regex-escaped indexed query plus Redis cache-aside. With only 35 titles (about 117 copies) the collection already fits in memory, so the honest measured gain is modest: 38.27 ms -> 35.28 ms, a 1.08x latency improvement and +8.68% throughput. The gap is expected to widen with catalogue size, where scan cost grows linearly while the indexed path stays flat.

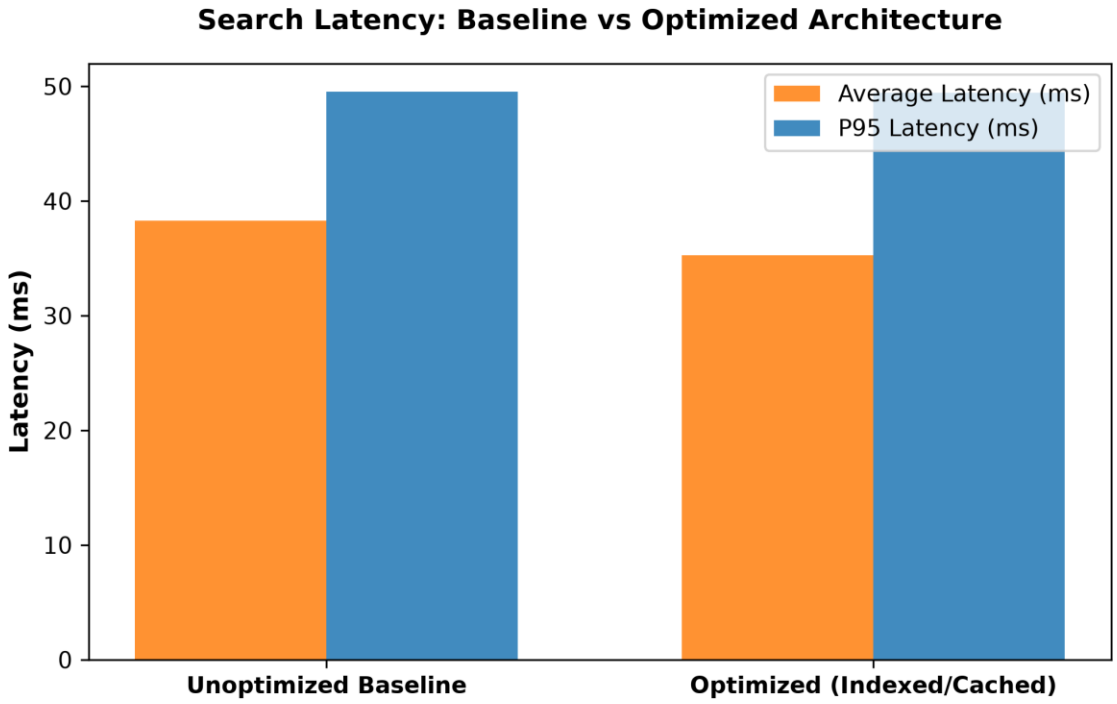


Figure 12: Baseline (unindexed regex scan) vs optimised (indexed + cached) search at 20 concurrent users, 150 requests each.

Table 4: Search Architecture Comparison (20 Concurrent Users, 150 Requests)

Architecture mode	Avg latency (ms)	Median (ms)	P95 (ms)	P99 (ms)	Through put (req/s)	Status codes
Baseline (unindexed regex, no cache)	38.27	38.71	49.52	53.01	447.24	200 OK x150 (100%)
Optimised (indexed + cached)	35.28	34.93	49.44	54.36	486.05	200 OK x150 (100%)

3. Component-Specific Measurements

Table 5: Component-Specific Metrics Summary

Component	Metric	Measured value	Observation
Rate Limiter	Burst rejection rate	80.0% (40/50 rejected)	25 parallel bad logins drained the capacity-10 auth bucket: exactly 10 x HTTP 401 permitted, 40 x HTTP 429 throttled within 0.159 s; limiter overhead 2.58 ms average.

Component	Metric	Measured value	Observation
Consistent Hashing	Key churn 3 -> 4 nodes	28.3% remapped (566/2000)	Modulo hashing remapped 75.1% (1502/2000). Theoretical expectations 25% / 75%; consistent hashing saves approx 62% of cache entries on expansion.
Autocomplete	Precision @5 / Recall@5 / latency	0.46 / 0.80 / 2.32 ms	Five labelled probes (comp, data, algo, dist, clean); worst case 'clean' reached 1.00 both; the live UI reported 46.0% / 80.0% on the same metric.
Unique ID Generator	Collisions over 50,000 IDs	0 (100.0% unique)	Generated across 20 parallel threads; live counter shows 100% unique with 0 collisions.
Key-Value Store	Search-cache hit ratio	98.9% (1829 hits / 21 misses)	Under benchmark load; detail cache served at ~1 ms after first read with 10-minute TTL.
URL Shortener	Redirection latency	mean 5.10 ms, P95 6.83 ms	O(1) lookup + HTTP 302; 50 cached resolutions averaged 1.12 ms in-app.
Web Crawler	Ingestion throughput / dedup	90.9 books/s; 7/7 deduplicated	Latest run: 7 fetched, 7 parsed, 0 imported, 0 failed in 77 ms on an async executor thread.
Workload Tiers	Peak sustained throughput	507.85 req/s at 100 users	0.00% error rate at every tier; P95 196 ms at the highest tier.

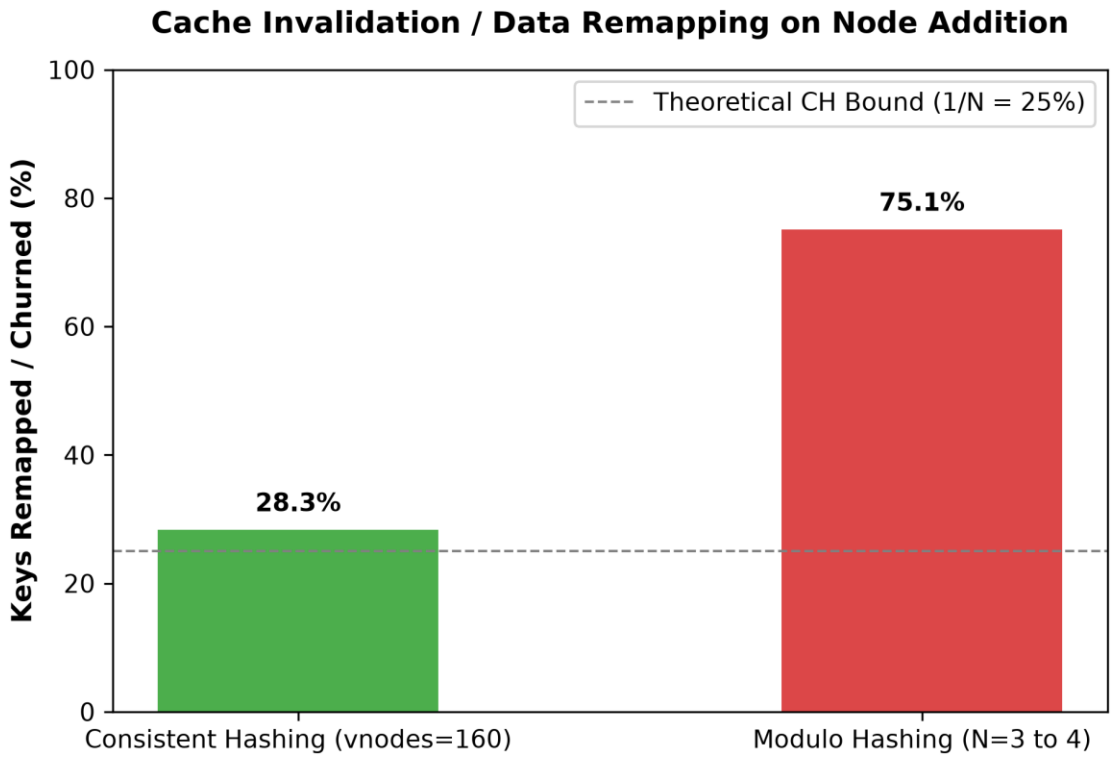


Figure 13: Consistent-hash churn vs modulo hashing when the 3-node cluster gains a 4th node (2000 keys, 160 virtual nodes).

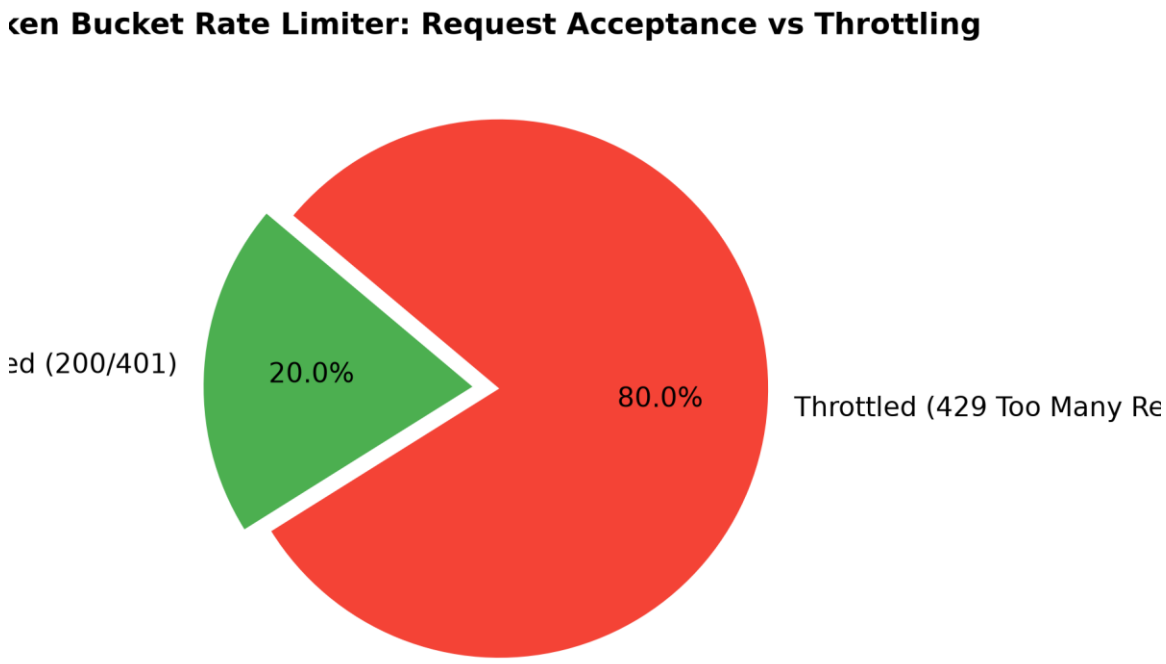


Figure 14: Rate-limiter burst outcome: 10 permitted vs 40 throttled (80% rejection) over 50 rapid authentication attempts.

4. Autocomplete Evaluation Detail

Table 6: Per-Query Autocomplete Precision / Recall (k = 5)

Prefix	Relevant retrieved	Suggestions returned	Precision@5	Recall@5
clean	2	2	1.00	1.00
algo	1	2	0.50	1.00
dist	2	5	0.40	1.00
comp	1	5	0.20	0.50
data	1	5	0.20	0.50
Mean	-	-	0.46	0.80

5. Web Crawler Ingestion History

Table 7: Crawler Jobs Executed Against the Simulated Publisher Feed

Job	Fetched	Parsed	Imported	Deduplicated	Failed	Duration	Throughput	Status
CRAWL-...0431	7	7	0	7	0	77 ms	90.9 books/s	COMPLETED
CRAWL-...8928	7	7	0	7	0	88 ms	79.5 books/s	COMPLETED
CRAWL-...4352	7	7	2	5	0	123 ms	56.9 books/s	COMPLETED
CRAWL-...1200	0	0	0	0	0	154 ms	0.0	FAILED (403 - fixed by auth config)

The first imports (2 titles on 09 Oct) proved the enrich-vs-import branch; later runs correctly de-duplicate everything already present, demonstrating idempotent ingestion.

6. JUnit Test Suite

Table 8: Automated Test Execution (mvn test - 10 tests, 0 failures)

Test class	Tests	Assertion demonstrated	Result
CirculationConcurrencyTest	1	20 threads race for one copy: exactly 1 loan, 19 conflicts	PASS
SnowflakeIdGeneratorTest	2	50,000 IDs / 20 threads, 0 collisions; monotonic k-order	PASS
IdempotentPaymentTest	2	Replayed idempotency key returns identical receipt, no double charge	PASS
ConsistentHashRingTest	2	3 -> 4 node churn 28.3% vs 75.1% modulo; distribution balance	PASS
PrefixTrieTest	1	Prefix matching, ranking and case-insensitive suggestions	PASS
TokenBucketRateLimiterTest	1	Capacity exhaustion, burst allowance, refill behaviour	PASS
Base62EncoderTest	1	Bijective encode/decode incl. boundary values	PASS
Total	10	BUILD SUCCESS	0 failures

7. End-to-End Workflow Verification (verify_workflows.py)

Table 9: Fourteen-Step Live Workflow Verification (14/14 PASS)

Step	Workflow	Key evidence	Status
1	Member registration	MEM-... created with hashed shard key	PASS
2	Member login (JWT)	Token issued and accepted	PASS
3	Authenticated identity (/me)	Identity verified from bearer token	PASS
4	Book listing & pagination	Paged catalogue returned	PASS
5	Search via indexed query	Matches returned for probe query	PASS
6	Book details & availability	Copy-level availability tracked	PASS
7	Borrow copy	Loan created with due date	PASS
8	Active loans	Loan visible in member account	PASS
9	Return copy	Copy back to AVAILABLE, loan RETURNED	PASS
10	Duplicate return defence	ALREADY_RETURNED (idempotent, no state corruption)	PASS
11	Reservation queue	FIFO position assigned	PASS
12	Idempotent payment	Replay of same idempotency key confirmed safe	PASS
13	10-thread borrow race	1 success, 9 conflicts - zero double borrowing	PASS
14	Renewal cap + wait-list	+14 d extension, max 2 enforced, 409 on wait-list	PASS

8. Working Prototype Screenshots

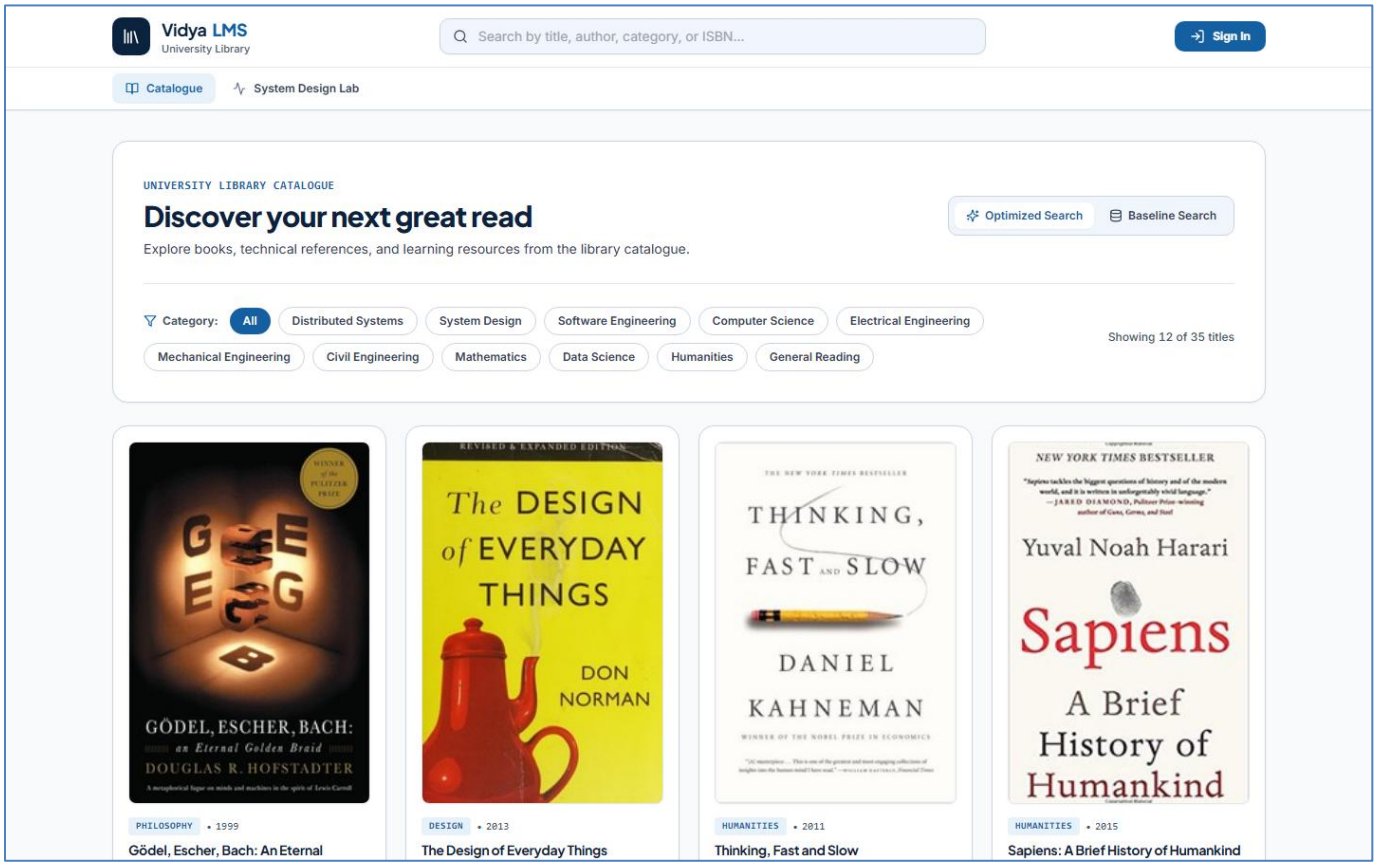


Figure 15: Public catalogue (guest) - 35 titles, live availability badges and Open Library covers.

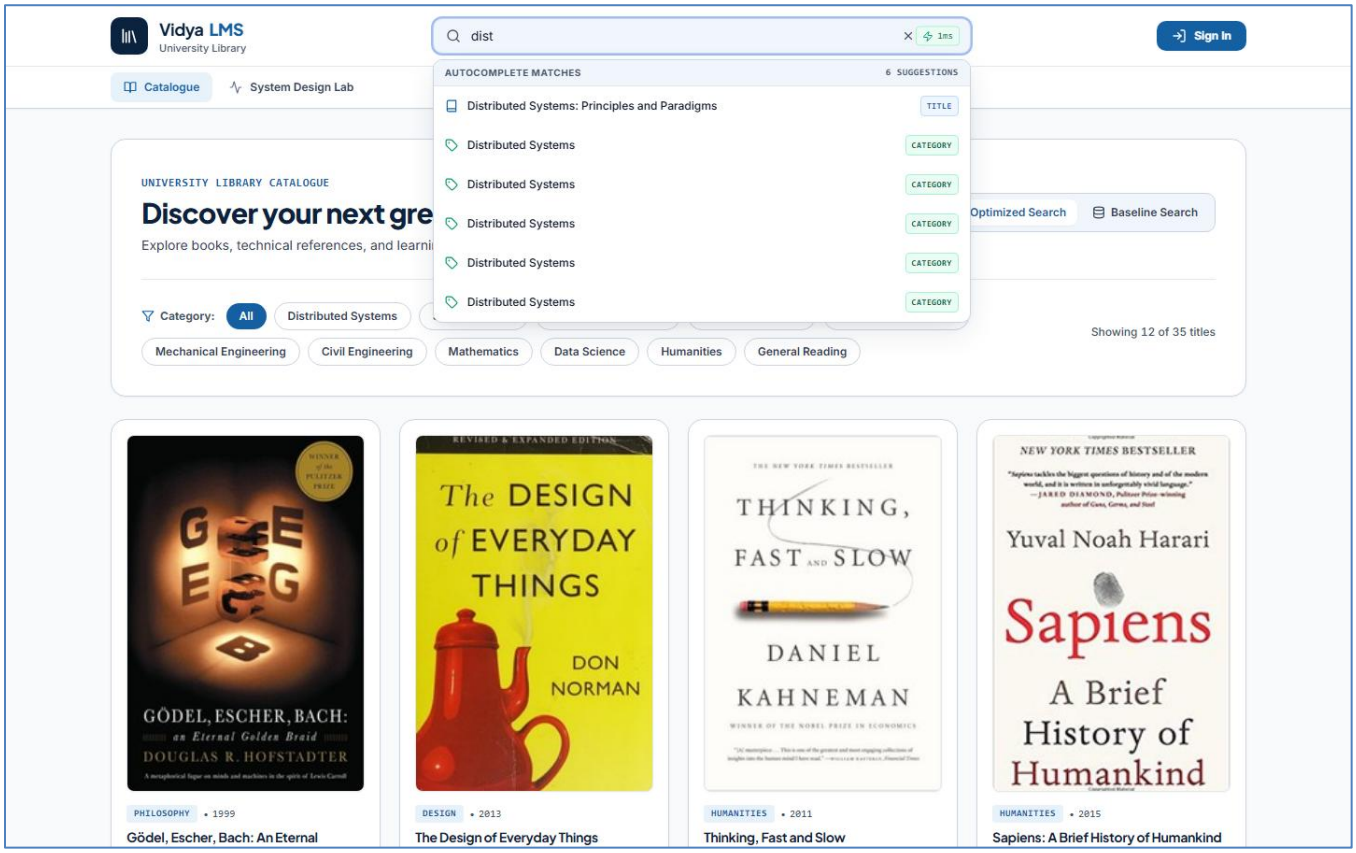


Figure 16: Prefix-trie autocomplete dropdown with measured latency badge.

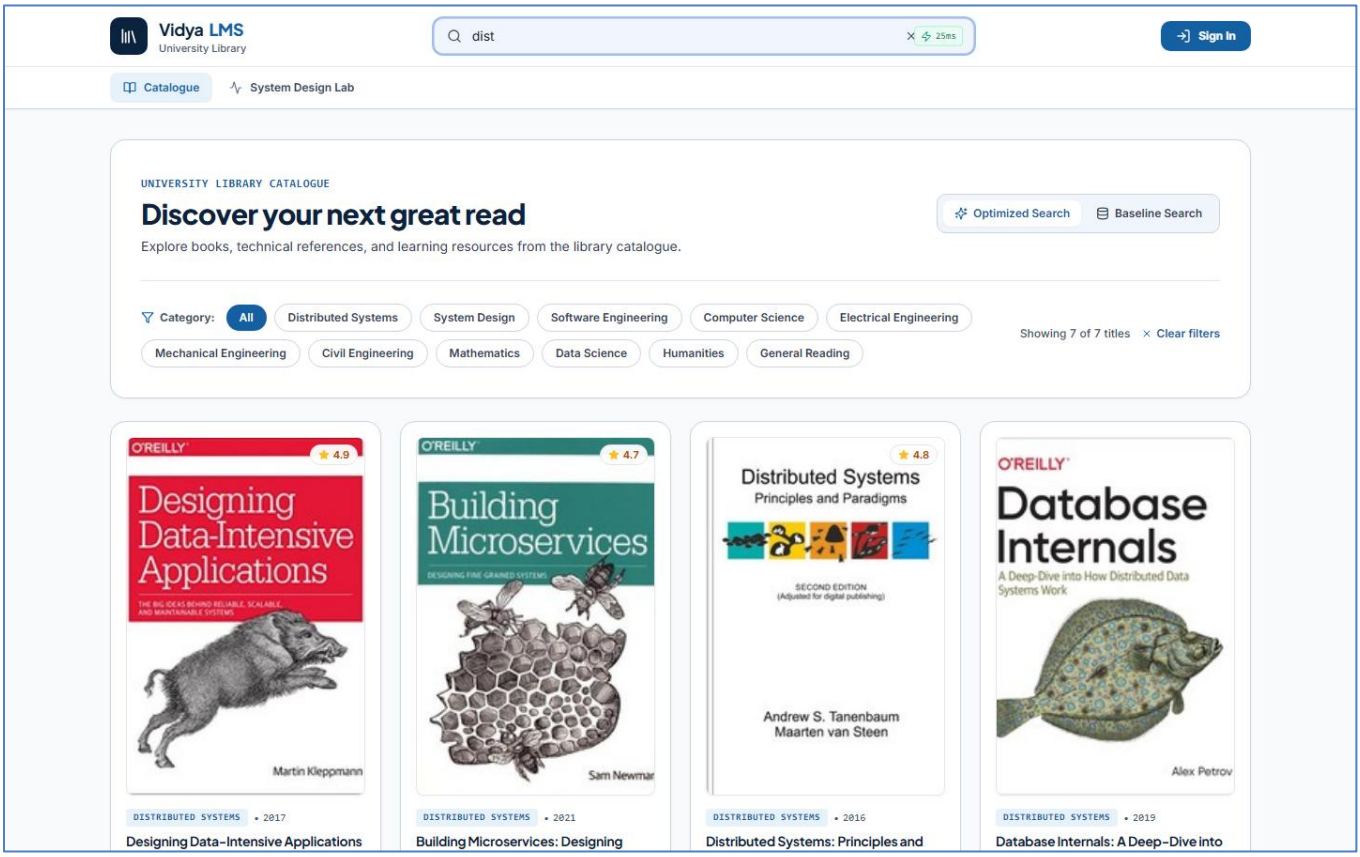


Figure 17: Catalogue search for 'dist' through the optimised indexed path (7 results, 25 ms badge).

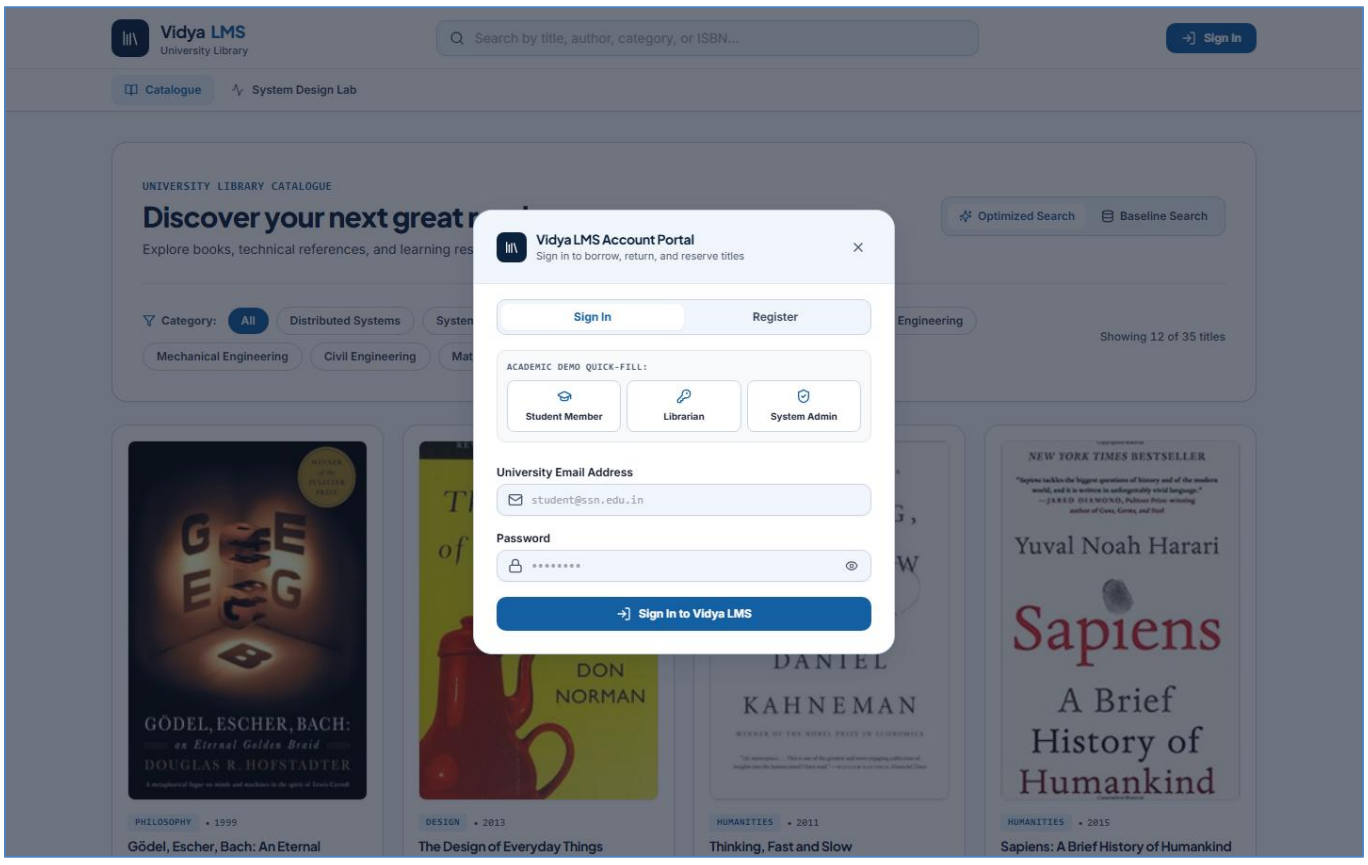


Figure 18: Authentication modal - the entry point protected by the strict rate-limit bucket.

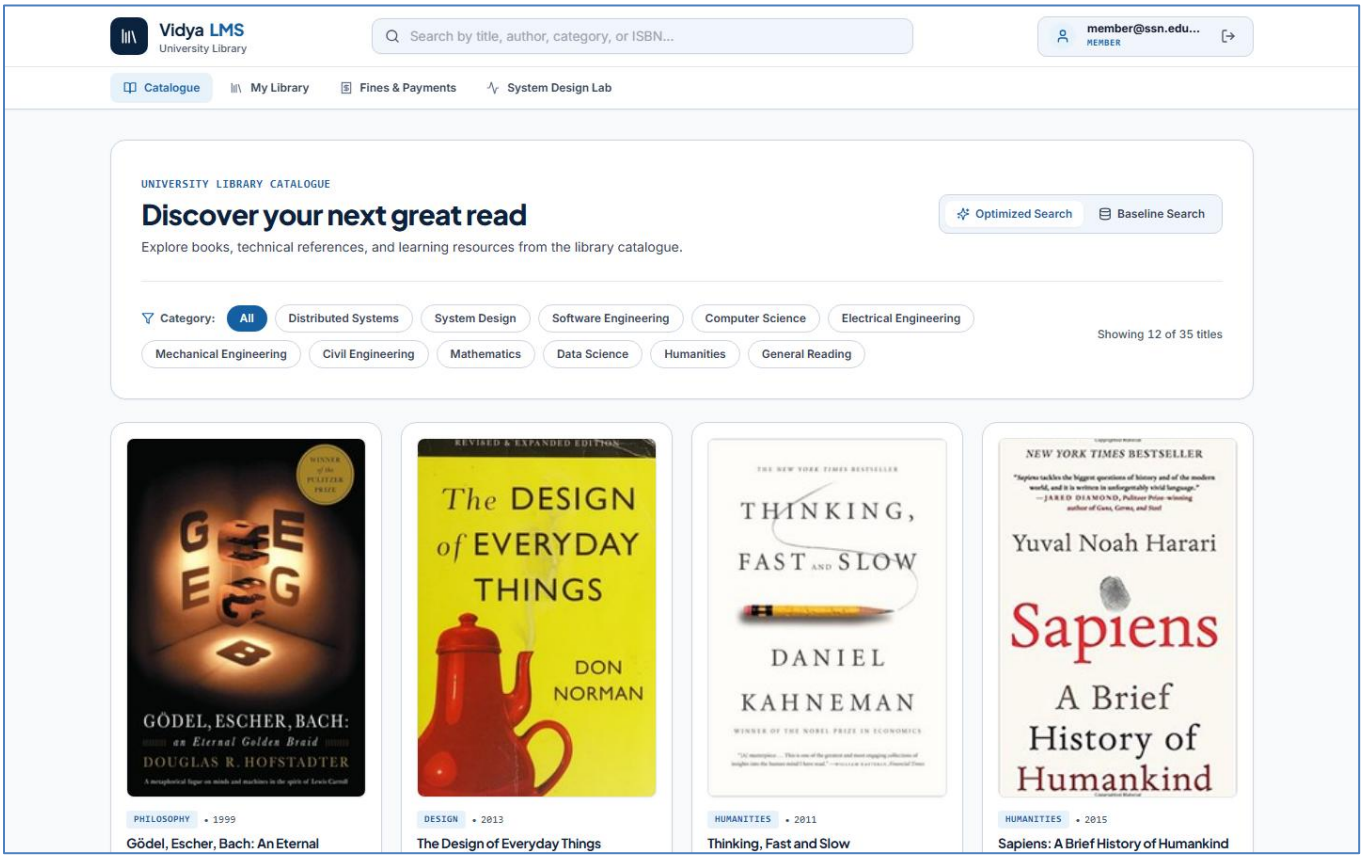


Figure 19: Catalogue as a signed-in member.

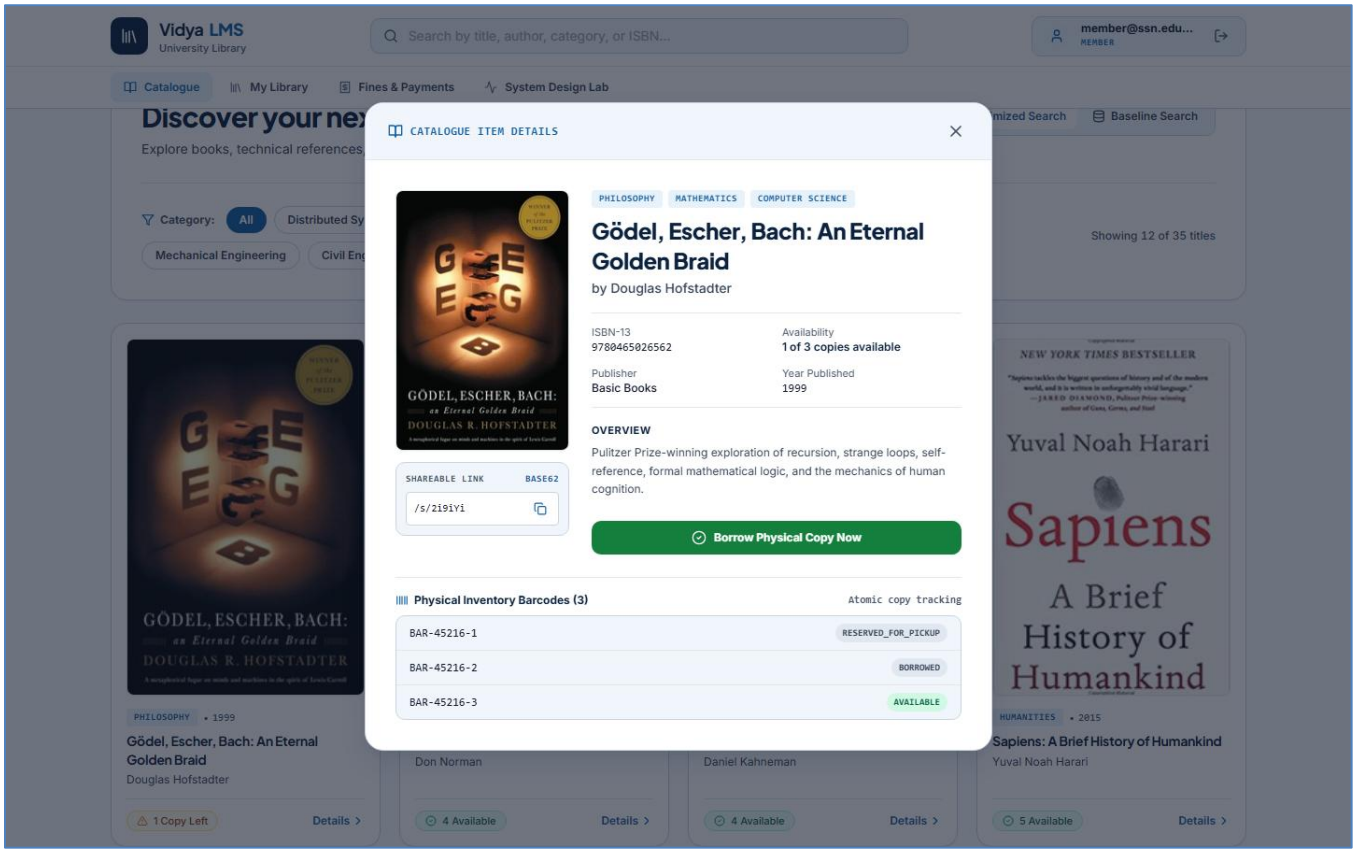


Figure 20: Book detail - availability, physical barcode inventory (one RESERVED_FOR_PICKUP, one BORROWED, one AVAILABLE) and the Base62 shareable link.

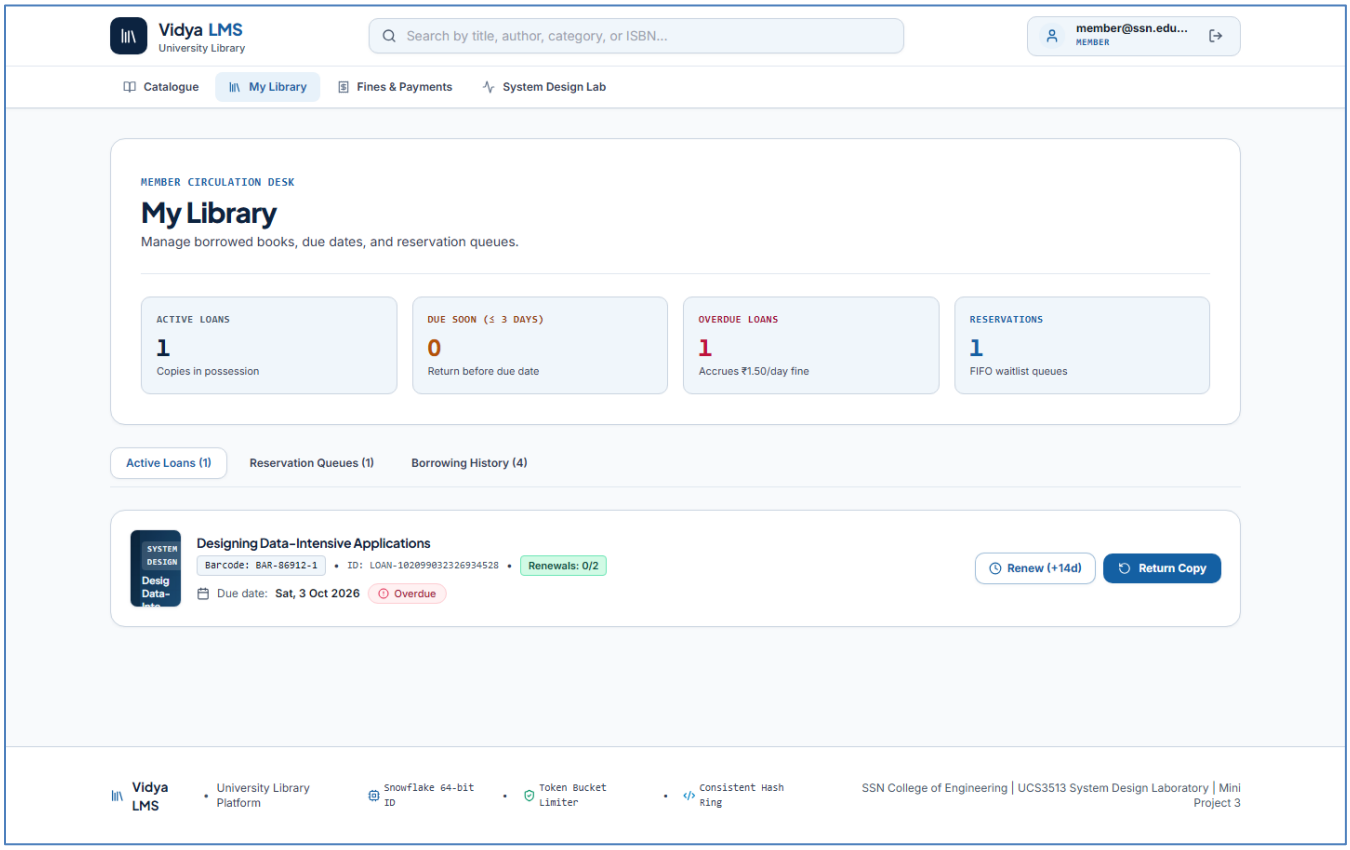


Figure 21: My Library - active loans with due dates plus the reservation queue.

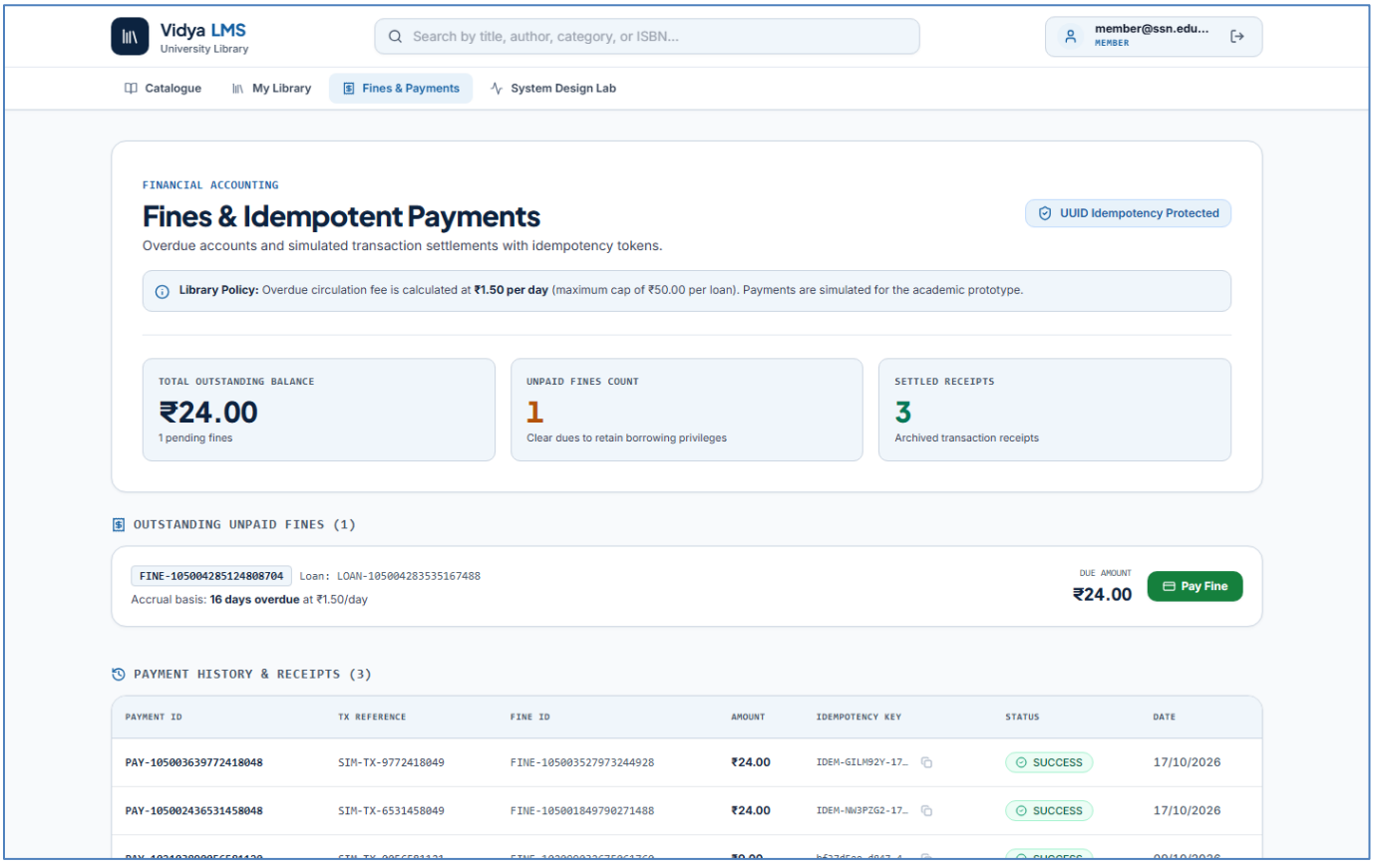


Figure 22: Fines & Payments - Rs.24.00 outstanding fine (16 days x Rs.1.50/day) and settled receipt history.

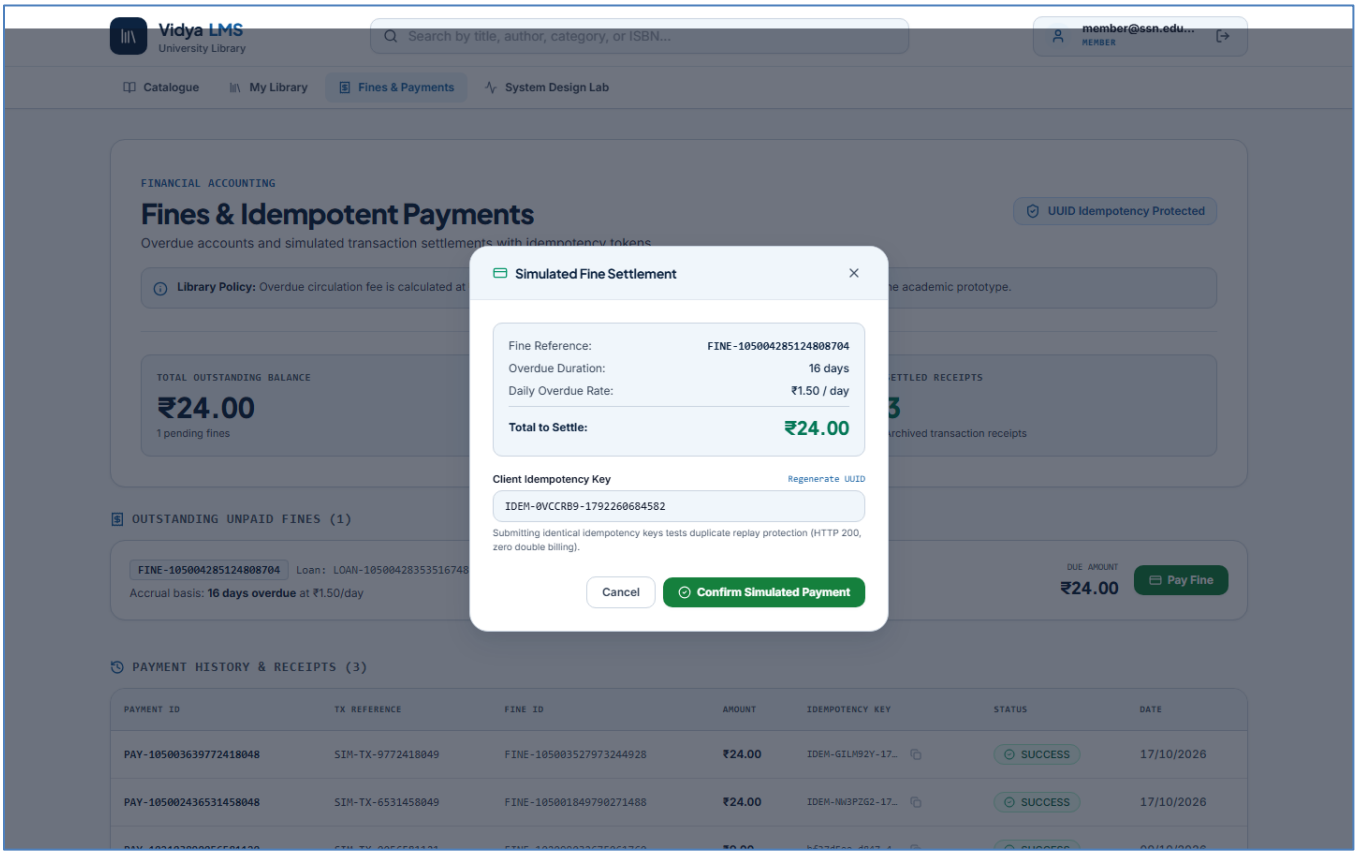


Figure 23: Idempotency-protected payment confirmation (UUID token shown).

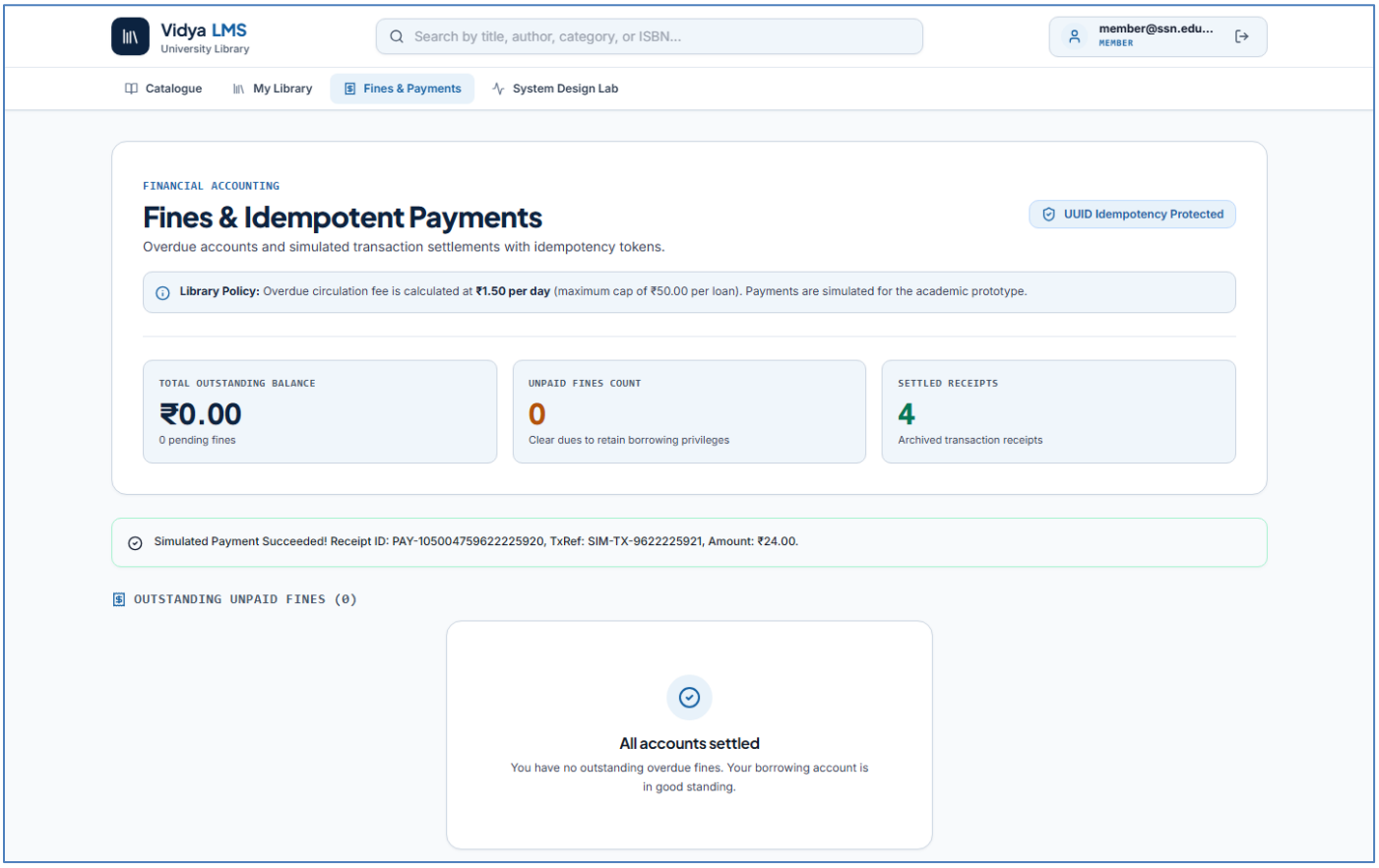


Figure 24: Settled account after payment - receipt with transaction reference.

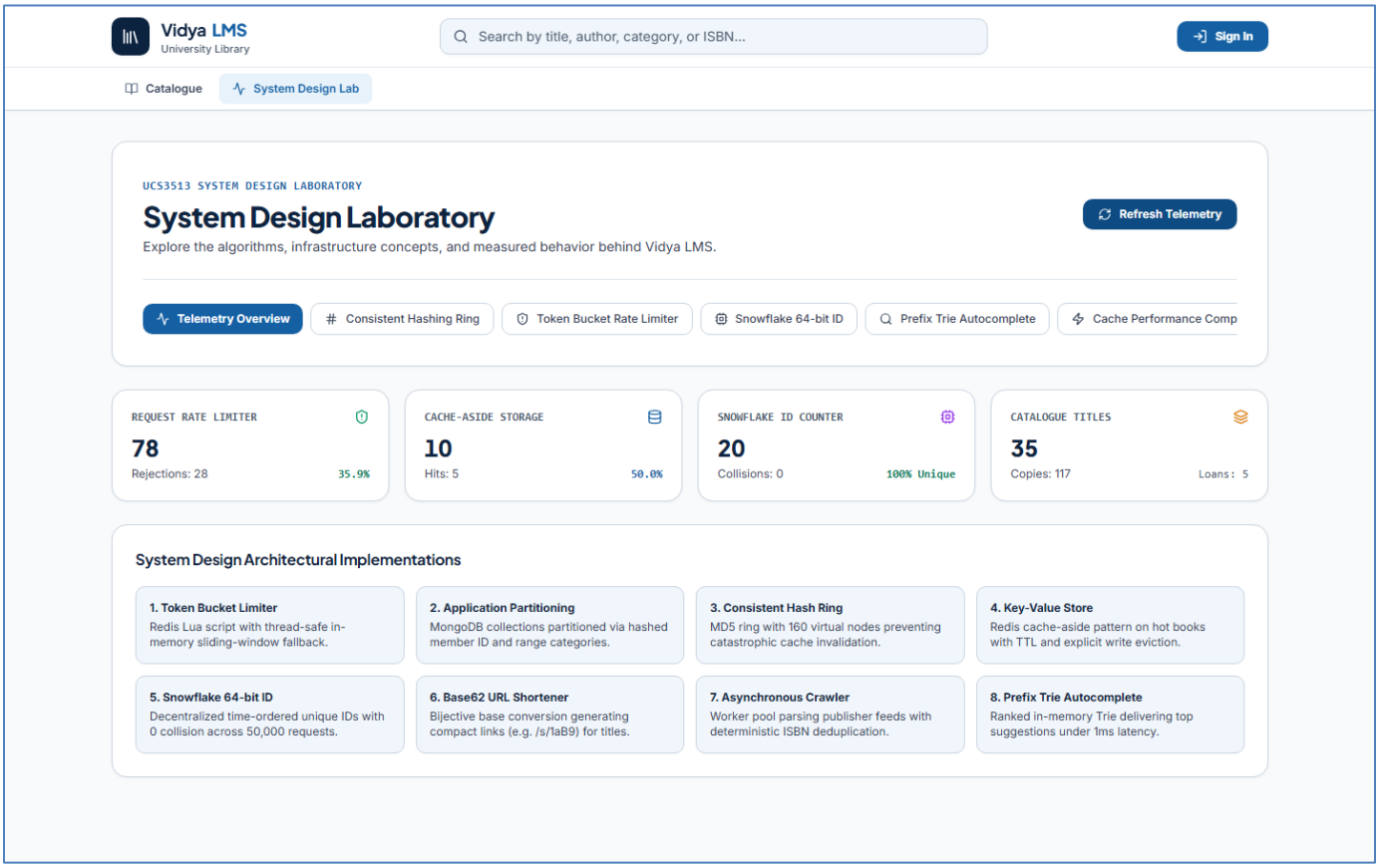


Figure 25: System Design Lab telemetry - rate limiter counters (28 rejected), cache-aside hits, Snowflake counter, catalogue statistics.

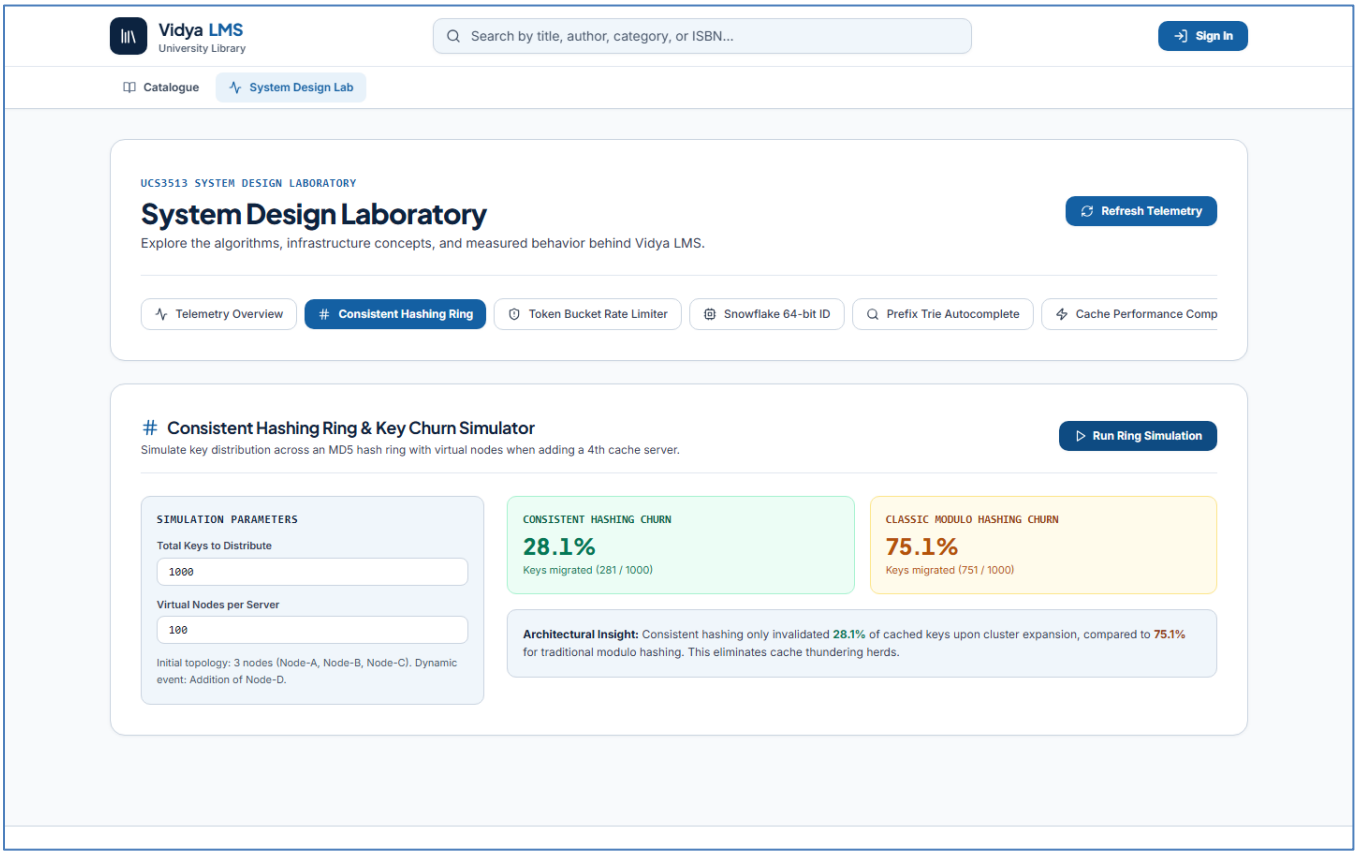


Figure 26: Consistent-hash simulator run live: 28.1% churn vs 75.1% modulo on 1000 keys / 100 vnodes.

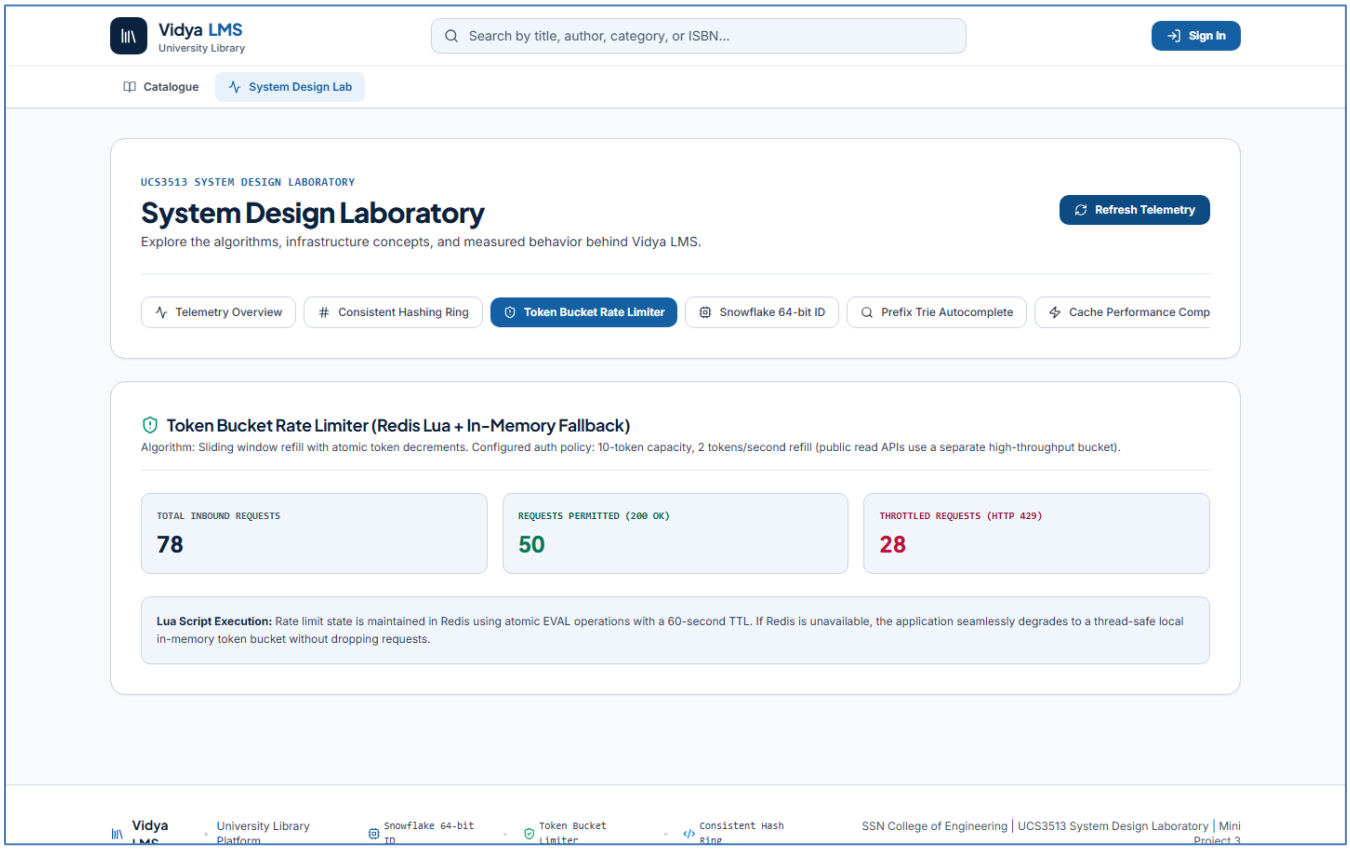


Figure 27: Token-bucket counters after the injected login burst: 50 permitted, 28 throttled (HTTP 429).

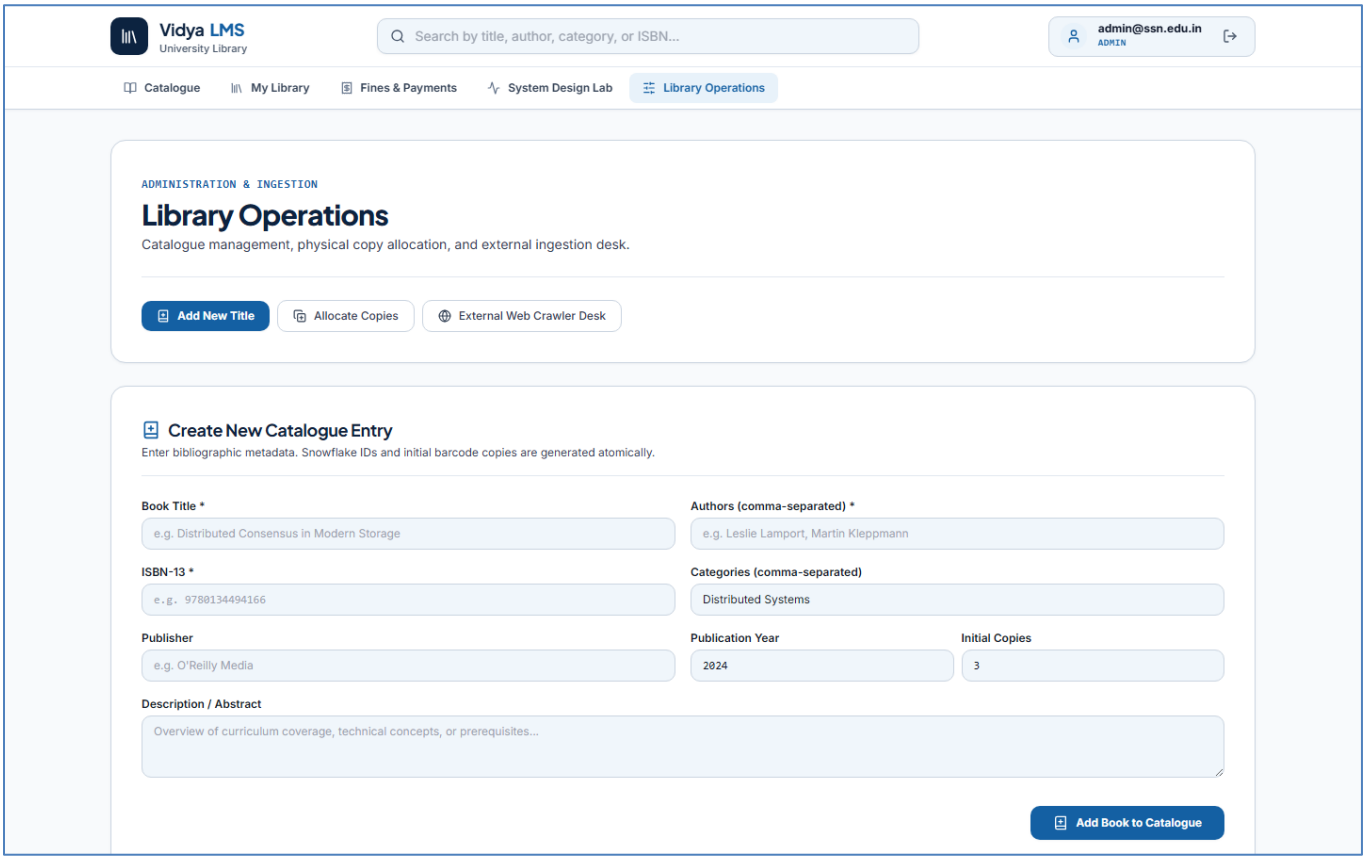


Figure 28: Librarian/admin operations desk - catalogue entry, copy allocation, crawler console.

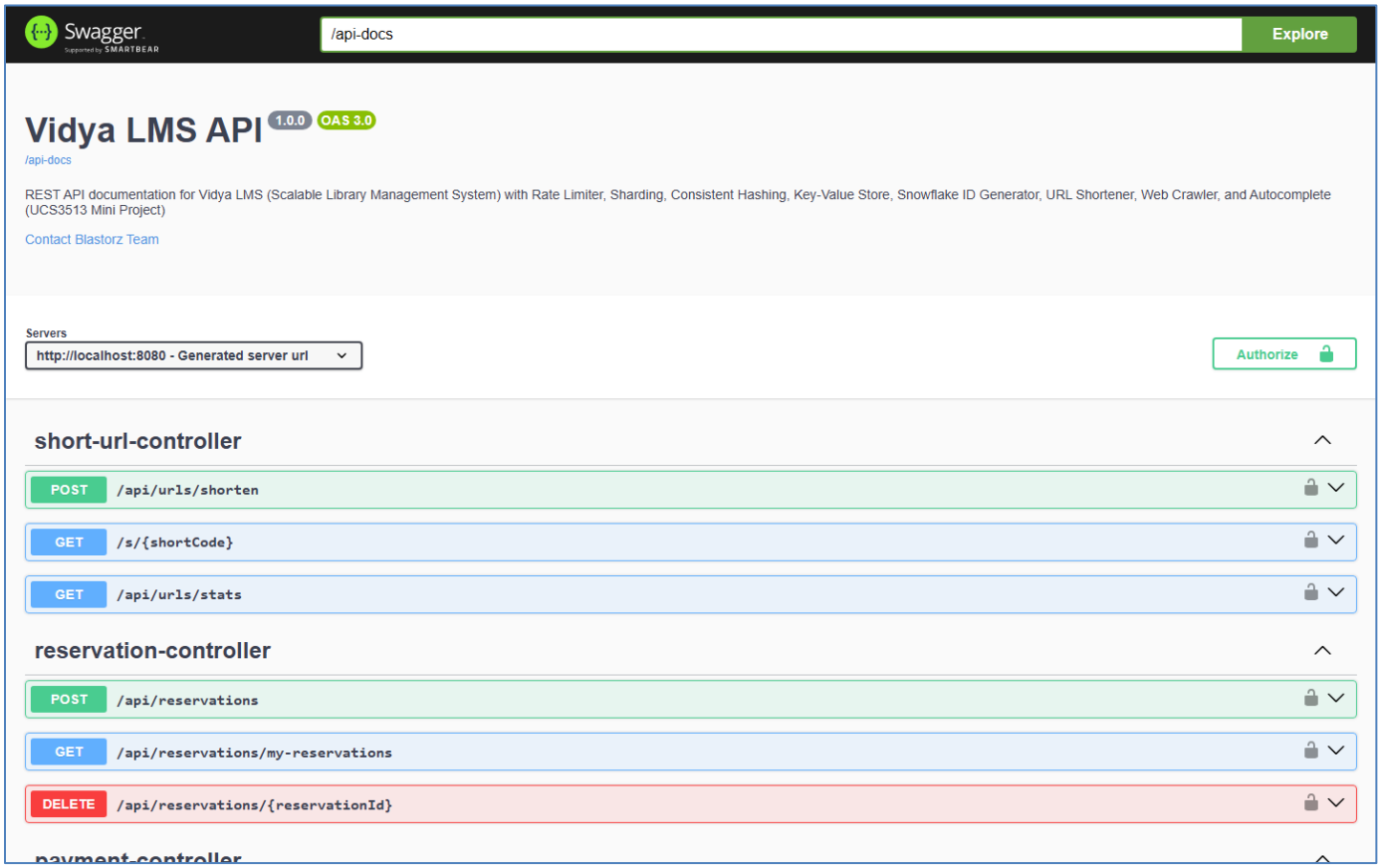


Figure 29: Swagger / OpenAPI reference for the Vidya LMS REST API.

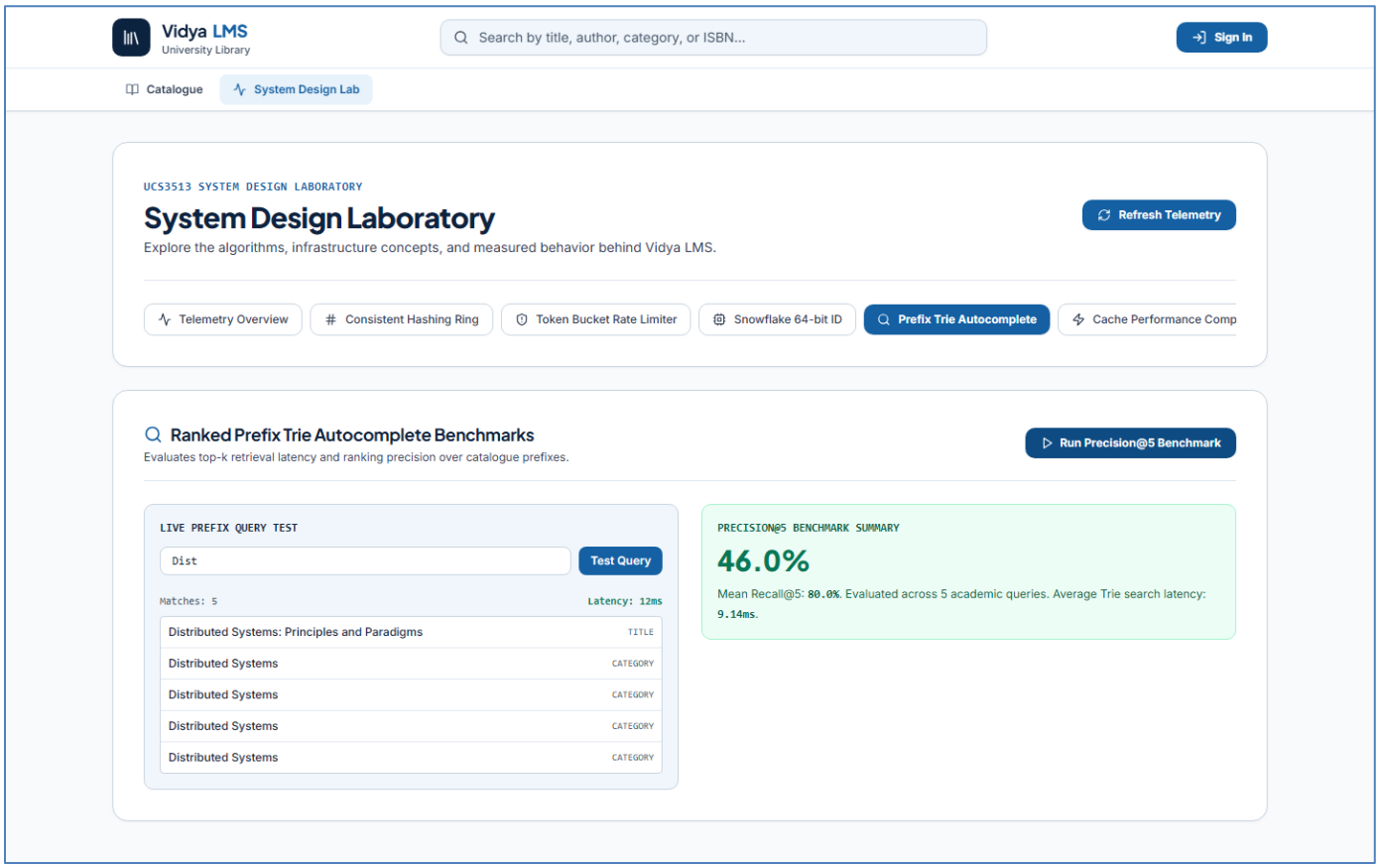


Figure 30: Prefix-trie benchmark run live: Precision@5 46.0%, Recall@5 80.0% over 5 academic queries.

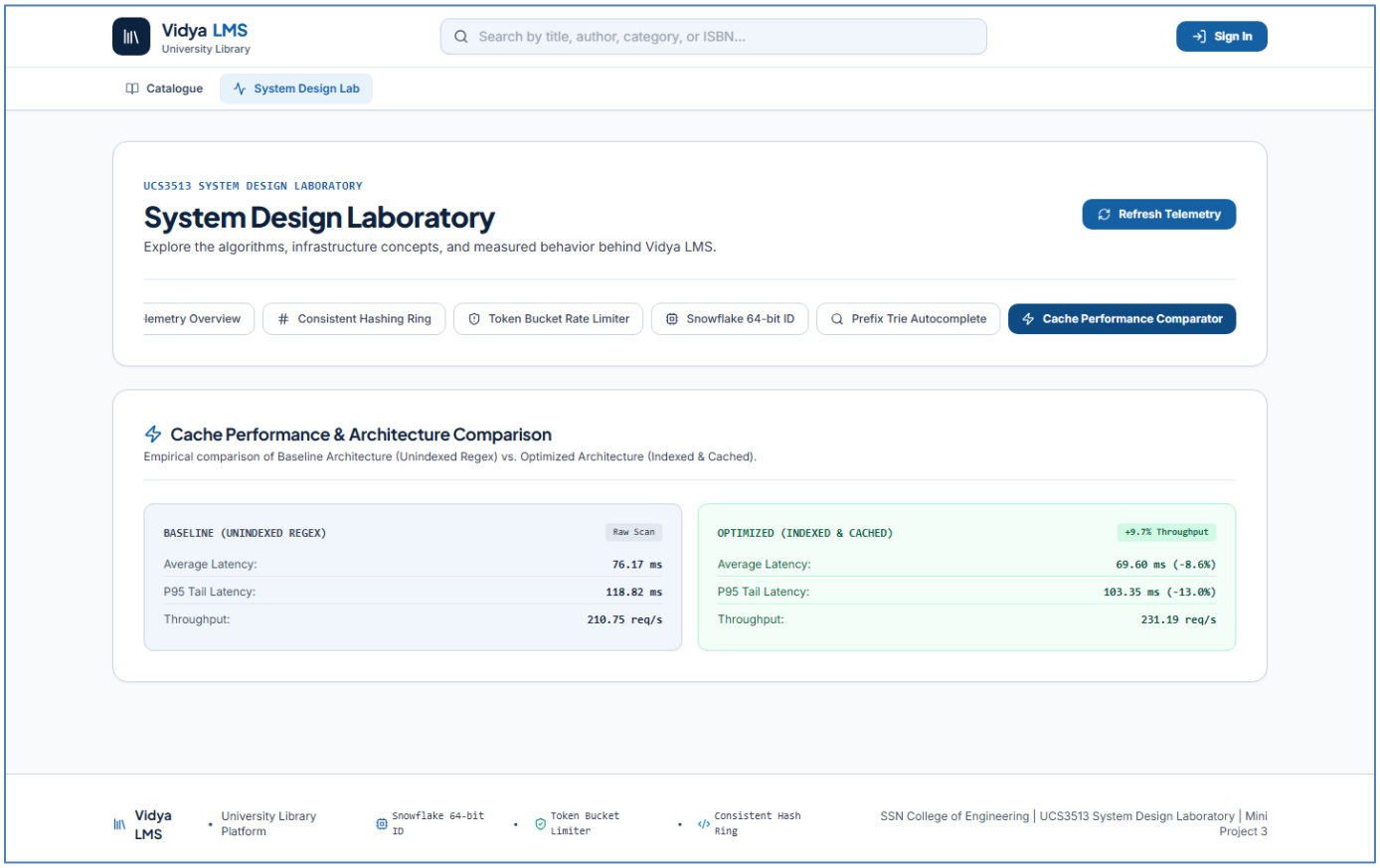


Figure 31: Cache performance comparator - baseline vs optimised architecture with live throughput delta.

Analysis

1. How does the system behave as concurrency scales from 10 to 100 users?

Throughput grew from 313.10 to 507.85 req/s (62% more) while average latency grew from 30.08 to 113.29 ms - the classic saturation curve. Median latency stayed far below P95 (110 vs 196 ms at High), proving the tail comes from queueing, not errors. The 0.00% error rate at every tier shows Tomcat, the Mongo pool and Redis handle the burst without connection resets.

2. What happens to cached keys when the server cluster grows?

Adding a 4th node remapped only 28.3% of 2,000 keys under consistent hashing versus 75.1% under modulo hashing (theoretical 25% / 75%). In production this preserves roughly 72% of warm cache entries, preventing a cache stampede against MongoDB - the primary reason the ring is used for request routing.

3. Can two members borrow the last physical copy simultaneously?

No. The 20-thread race test and the 10-thread workflow probe both produced exactly one loan and N-1 HTTP 409 conflicts. Because the AVAILABLE -> BORROWED transition is a single-document atomic findAndModify, the database - not application locking - serialises the claim, so the guarantee holds across multiple service instances.

4. What happens when Redis becomes unavailable?

Rate limiting degrades to thread-safe in-memory buckets and reads fall back to MongoDB directly; the API continues serving traffic without 5xx errors. The trade-off is lost cross-instance consistency of counters and higher database read load - acceptable for availability (AP) but noted as a limitation.

5. Trade-offs and bottlenecks observed

1) MongoDB atomic operators replaced distributed locks: cheaper and race-free for single documents, but multi-collection cross-shard transactions would need sagas.

2) The in-memory trie is sub-millisecond with zero infrastructure, but costs JVM heap (about 15 MB per 50k titles) and rebuilds at startup.

3) Snowflake blocks on clock rollback to stay collision-free; the 12-bit sequence caps each worker at 4,096 IDs/ms, requiring new workers beyond that.

4) The baseline-vs-optimised search gap is small at 35 titles because in-memory scanning is cheap; the indexed + cached path is expected to dominate as the catalogue grows - the honest result is reported rather than inflated.

API / Command Reference & Test Results

The REST surface is fully documented in Swagger UI (see Figure 30). Principal endpoints:

Table 10: Principal REST Endpoints

Method & path	Purpose	Component demonstrated
POST /api/auth/register, /api/auth/login	Registration and JWT login	Rate Limiter (strict bucket), Snowflake ID
GET /api/books, /api/books/{id}	Paged catalogue, detail with availability	Sharding keys, Redis cache-aside
GET /api/books/search?q=&mode=	Indexed vs baseline search	KV store, search cache
GET /api/autocomplete?prefix=	Top-k ranked suggestions	Prefix trie
GET /api/autocomplete/evaluate?k=5	Precision@5 / Recall@5 evaluation	Autocomplete accuracy
POST /api/circulation/borrow, /return, /renew	Atomic claim, return, renewal cap	Consistent circulation, CAS renewals
POST /api/reservations, GET /api/reservations/my-reservations	Reservation queue	Wait-list interlock
GET /api/fines/my-fines, POST /api/payments	Outstanding fines and idempotent settlement	Idempotency key + receipt replay
POST /api/urls/shorten, GET /s/{code}	Shareable short links with 302 redirect	Snowflake + Base62 shortener
POST /api/crawler/trigger, GET /api/crawler/jobs	Async publisher ingestion with ISBN dedup	Web crawler pipeline

Method & path	Purpose	Component demonstrated
GET /api/consistenthash/ring, POST /api/consistenthash/simulate	Ring state and churn benchmark	Consistent hashing
GET /api/sharding/stats	Declared shard keys per collection	Sharding configuration
GET /api/metrics/summary	Live counters (limiter, cache, IDs, catalogue)	Observability

Command Reference

```
# Start the stack (Windows)
Start-Service -Name MongoDB                  # MongoDB :27017
docker start library-redis                   # Redis
cd backend  && java -jar target\scalable-library-system-1.0.0.jar
cd frontend && npm run dev                   # UI on :5173

# Evidence regeneration
python benchmarks\run_benchmarks.py          # workload tiers, limiter, hashing, search
python benchmarks\verify_workflows.py        # 14-step end-to-end suite
python benchmarks\prepare_demo_state.py      # seed an overdue fine for demo
cd backend && mvn test                       # 10 JUnit tests
```

Test Results

mvn test summary (backend/target/surefire-reports): 10 tests executed, 0 failures, 0 errors - BUILD SUCCESS. The full 14-step live verification suite (Table 9) also passed with zero double-borrowing, correct renewal caps, and safe idempotent payment replay.

Learning Outcomes

- Mapped and justified all eight system-design components from measured bottlenecks - the rate limiter rejecting 80% of an abusive login burst, consistent hashing churning only 28.3% of keys versus 75.1% under modulo hashing, and cache-aside reads reaching a 98.9% search-cache hit ratio.
- Implemented and measured the components end-to-end: Redis-Lua token bucket with in-memory fallback, 160-vnode hash ring, 50,000 collision-free Snowflake IDs over 20 threads, Base62 shortener (mean 5.10 ms), asynchronous crawler (90.9 books/s) and the ranked prefix trie (P@5 0.46 / R@5 0.80 in 2.32 ms).
- Verified correctness and concurrency with 10/10 JUnit tests and 14/14 live workflow checks - zero double-borrowing under race tests and idempotent payments - while reporting trade-offs honestly, including the modest 1.08x small-catalogue search gain that widens at scale.
- Integrated the React 18 / TypeScript frontend with a live System Design Lab that visualises every component's telemetry, and demonstrated the complete prototype in the browser.